IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 16 January 2026, accepted 21 January 2026, date of publication 30 January 2026, date of current version 2 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3659713

SURVEY

Generative Artificial Intelligence for Software Development Using ISO/IEC 29110 Basic Profile: Gaps and Opportunities

JOSE VALDIVIA[1], DAVID MAURICIO[1], AND ROSARIO GUZMÁN-JIMÉNEZ[2], (Senior Member, IEEE)

[1]Faculty of Systems and Informatics Engineering, Universidad Nacional Mayor de San Marcos, Lima 15081, Peru
[2]Program Systems Engineering, Department of Engineering, Universidad de Lima, Lima 15023, Peru

Corresponding author: Rosario Guzmán-Jiménez (rguzman@ulima.edu.pe)

This work was supported by the Universidad Nacional Mayor de San Marcos–RR 005446-2025-R/UNMSM under Project C25202801.

ABSTRACT This study conducted a PRISMA-based systematic literature review of 52 primary studies published between 2022 and 2024 indexed in Scopus, Web of Science, and Springer. The objective of this study was to examine how generative artificial intelligence can be applied to software development in small and medium-sized software development enterprises adopting the ISO/IEC 29110 Basic Profile. This review identified six predominant generative artificial intelligence methods (e.g., large language models and prompt engineering), 32 key tools (e.g., ChatGPT and GitHub Copilot), five metric categories (efficiency, productivity, quality, precision, and user satisfaction), and 29 emerging roles (e.g., prompt engineering and machine learning engineers). The analysis reveals notable gaps in the application of generative artificial intelligence within specific process groups of the Basic Profile, particularly in Project Management (Project Plan Execution, Project Assessment and Control, and Project Closure) and Software Implementation (Implementation Initiation and Product Assembly). To address these gaps, this study proposes a structured alignment framework that maps the identified generative artificial intelligence methods, tools, metrics, and roles to the requirements of the ISO/IEC 29110 Basic Profile. Derived from a systematic synthesis of the literature, this framework highlights opportunities for generative artificial intelligence that support standard-compliant adoption across the software development lifecycle. This provides actionable guidance to small- and medium-sized software development enterprises. Future research directions include empirical validation through industrial case studies and the integration of software quality assurance mechanisms within generative artificial intelligence-enabled processes.

INDEX TERMS Software engineering, generative artificial intelligence, software development processes, software tools and metrics, software roles.

I. INTRODUCTION

Artificial intelligence (AI) is a pivotal technological innovation that drives the transition from Industry 4.0 to Industry 5.0. This paradigm allows for the development of adaptive, synergistic, and ethically aware, or morally consistent systems. Prescriptive and intelligent systems have demonstrated potential to enhance creativity, resilience, and operational efficiency in complex environments. Furthermore, the convergence of technologies (i.e., blockchain, edge computing, robots, big data analytics, and the Internet of Things enabled by 6G) has accelerated the emergence of global smart ecosystems. Industry 4.0 places emphasis the digitization of the production process through cyber-physical systems, the Internet of Things, and the Internet of Services [1]. Industry 5.0 extends this paradigm by emphasizing the responsible use of public values and sustainable human development [2], [3].

Generative artificial intelligence (GAI) has the potential to transform multiple domains, including manufacturing, health, and finance, by improving the efficiency and adaptability of professionals [4], [5]. The accelerated evolution

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

of this process has had a disruptive effect on the established workflow and development practices. In the software development (SD) domain, GAI-based tools have shown significant advantages in terms of productivity, automation, and process quality. Recent studies [6], [7] have demonstrated that systems such as ChatGPT and GitHub Copilot can help automate tasks, improve workflow, and reduce the learning curve for coding. These technologies complement traditional coding and project management (PM) practices, enabling shorter development cycles in agile environments and improving collaboration and decision-making processes [8]. Furthermore, empirical evidence demonstrates that GAI-based tools accelerate key activities such as code completion, design prototypes, requirement validation, documentation generation, and the generation of more specific solutions [9].

Within SD, GAI is increasingly employed to replace traditional methods, and is applied across nearly all phases of the software development life cycle (SDLC) in software projects, including planning, design, implementation, and testing. Furthermore, GAI-based tools can facilitate development of high-quality and robust software systems, thereby enhancing delivery capabilities in the face of technological evolution and competitive pressures [10], [11], [12].

These SD methods have an agile focus, and improve process efficiency, quality, and adaptability. These can be adapted to the SDLC of various projects and development teams. [13], [14]. This approach emphasizes the utilization of tools and concepts designed to simplify the process of planning, monitoring, and control, thereby enhancing the success and satisfaction levels of the subjects involved [15].

The integration of GAI into the SD process facilitates the automation of repetitive tasks such as testing, debugging, and code generation. This capacity is especially advantageous for small- and medium-sized software development enterprises as it facilitates the adoption of quality standards, including the ISO/IEC 29110 Basic Profile. The implementation of automation in routine activities is a strategy employed by GAI to reduce operational overhead and support the efficient and scalable implementation of standardized processes.

Furthermore, GAI has demonstrated the ability to save time and effort while improving software quality by analyzing large datasets to detect defects, inconsistencies, and faulty logic. It also supports the development of innovative solutions and user interface designs, contributing to improved usability and overall system performance [16].

II. METHODS, TOOLS, METRICS, AND ROLES

GAI influences SD by transforming the methods, coding practices, testing activities, and software evaluation processes. The development of applications supported by the GAI involves the use of diverse methods and tools. A structural method provides a framework for managing processes and risk while enabling the identification of approaches such as the Large Language Model (LLM) and deep learning techniques.

A. SD CONCEPTS AND THE GAI

Recent studies have indicated that the integration of AI with Test-Driven Development is a promising approach for enhancing the effectiveness of systematic code development and validation [17]. In domain software testing, AI-driven approaches improve bug detection and test -case development, thereby enhancing the creation of more reliable solutions [9]. It has been demonstrated that GAI-based tools, such as the GitHub Copilot and LLM, including ChatGPT and Codex, support automated code generation and assistance. These advancements have been shown to improve development speed and efficiency [6], [8], [18], [19], [20], resulting in improved software quality, optimized resource utilization, and increased agility.

Metrics and roles play a fundamental role in SD, as supported by GAI. The utilization of these metrics enables the establishment of an objective for evaluating the quality of AI-generated codes, including their accuracy and efficiency. These measures empower developers to evaluate the congruence between the resulting artifacts and original objectives, thereby identifying opportunities for enhancement [21], [22]. More broadly, metrics support software engineering quality assessment by quantifying the key characteristics of the development process, including efficiency and maintainability. A review of the literature reveals that commonly reported metrics include structural complexity measures, which assess software architecture using indicators such as the number of modules and interconnections [23]. Quality metrics are used to forecast software reliability and support decision-making throughout the SDLC [24]. Finally, productivity metrics are reported, and these assess the impact of GAI-based tools on development speed, quality, and overall efficiency [6], [25].

The adoption of GAI in SD has transformed the professional roles and responsibilities. Developers and software engineers have an increasing emphasis on critical thinking, creativity, and the ability to solve complex problems. Concurrently, there has been the emergence of roles that prioritize the validation of AI-generated artifacts to ensure their correctness, safety, and compliance with project requirements [8], [26]. In the academic context, curricula are beginning to incorporate GAI-based tools that emphasize high-level design skills and problem decomposition [27], [28]. The presence of clearly defined team roles in GAI-supported development in conjunction with complementary skill sets and enhanced communication has been demonstrated to facilitate the achievement of SD objectives [29].

The applicability of the GAI has been demonstrated across a wide range of domains. These domains include, but are not limited to, the generation of creative content, such as poetry and music [30], as well as SD-related tasks such as natural language–to–code generation [31] and translation between programming languages [32]. However, these capabilities present several challenges. The AI-generated code must undergo rigorous validation to ensure correctness, performance efficiency, and appropriate resource utilization, particularly in production environments [33], [34], [35], [36], [37].

Despite the documented benefits of automated testing and coverage analysis in supporting defect detection and project management optimization [11], a substantial gap remains in the systematic application of GAI across the entire SDLC.

Existing GAI-based tools, including Copilot for code completion and LLM-based systems, such as ChatGPT for natural language-driven code and text generation [1], [22], [38], provide valuable support. Nevertheless, despite extensive training on large-scale datasets, contemporary LLMs demonstrate limitations in terms of accuracy and reliability when outputs are holistically evaluated [38]. A review of the literature reveals a lack of comprehensive frameworks that integrate GAI across all phases of the SDLC. This deficiency limits the potential of GAI to transform established practices. This gap underscores the necessity for more rigorous evaluation and integration frameworks that address both functional and nonfunctional aspects, including technical correctness, ethics, maintainability, and project constraints [19], [32].

Accordingly, GAI in SD signifies a paradigm shift that augments and reshapes the existing methods, tools, metrics, and roles. This transformation has the potential to improve operational efficiency and scope for innovative design, implementation, and maintenance. The GAI introduces novel development practices, evaluation mechanisms, and professional responsibilities that extend beyond traditional SD models.

To leverage GAI effectively and responsibly, it is essential to systematically integrate these capabilities to optimize development processes, improve technical quality, and enhance long-term software adaptability. The implementation of a structured approach for identifying opportunities is essential for supporting the transition toward more agile, accurate, and robust software engineering practices. This approach ensures that software products address both the current and future requirements, thereby contributing to the enhancement of software quality and reliability.

The objective of this study was to systematically identify and analyze the opportunities that GAI offers for the evolution of methods, tools, metrics, and roles in SD. A systematic literature review was conducted using Scopus, Web of Science and Springer databases. This review focuses on recent research addressing the application of GAI across SDLC. The resulting evidence was systematically organized and critically analyzed to formulate a framework for integrating GAI into the international standard ISO/IEC 29110 Basic Profile [39], with the aim of updating and improving its core processes. Although previous studies have explored AI in SD [11], [40], [41], there has been no explicit addressing of the gaps and opportunities associated with GAI adoption in relation to the specific requirements of this Basic Profile. This is particularly evident in the context of small- and medium-sized software development enterprises.

The contributions of this study are fourfold. First, it provides an overview of state-of-the-art SD practices supported by GAI. Second, it presents a structured and classified inventory of the emerging tools, processes, metrics, and roles. Third, it identified concrete opportunities for GAI adoption in each dimension. Finally, adaptations to the ISO/IEC 29110 Basic Profile process are proposed. These recommendations aim to facilitate the systematic integration of GAI capabilities, thereby contributing to the ongoing innovation in international software engineering standards.

The remainder of this paper is organized as follows. The objective of Section II is to introduce the fundamental concepts of GAI and SD. Section III delineates the research methodology and presents the results of the systematic literature review. Section IV discusses the GAI-based opportunities aligned with the ISO/IEC 29110 Basic Profile. This examination is accompanied by the proposition of an integration framework that aims to facilitate the seamless integration of these prospects. Section V discusses the findings and their implications, and Section VI concludes the study and outlines the practical implications and directions for future research.

B. BENEFITS OF GAI FOR SD

Structured software development refers to a systematic process in which software systems are designed, implemented, tested, and maintained through well-defined phases, standardized documentation, and predefined procedures. The utilization of SD methodologies guarantees that each development phase is executed in a controlled and predictable manner, thereby enhancing consistency, quality, and process reliability.

The emergence of GAI has significantly transformed this domain, predominantly through automation of routine and repetitive developmental activities. Automation of tasks, such as code generation, testing, and debugging has been demonstrated to result in accelerated development cycles and enhanced code quality. The widespread adoption of tools, such as ChatGPT, GitHub Copilot, and Tabnine, has profoundly transformed software engineering practices by providing developers with intelligent code suggestions, early error detection, and enhanced syntactic and semantic accuracy. Consequently, developers can transition their focus from repetitive tasks to higher-level design, reasoning, and complex problem-solving activities [42].

In addition to supporting new SD, GAI has demonstrated value in legacy system maintenance, software architecture improvement, and enhancement of the overall developer experience [16]. Nevertheless, the integration of GAI into SD processes presents several challenges, including variability in the output accuracy, potential model bias, and security vulnerabilities. Consequently, continuous human oversight remains essential for ensuring software quality, safety, and compliance with the development standards [11]. Notwithstanding the aforementioned limitations, GAI presents opportunities to improve both the efficiency and quality of the SD processes.

The main advantages of adopting GAI in SD are outlined as follows:

- Automation of repetitive tasks: GAI facilitates the automation of activities such as source code generation, testing, and documentation; accelerates the SDLC and

allows developers to concentrate on higher-level design and creative tasks [14].

- Improved code generation through implementation of prompt engineering (PE). Empirical studies have demonstrated that effective PE can result in more relevant and higher-quality code generation, while concurrently reducing errors and long-term maintenance costs [31].
- Support for design and performance optimization: GAI facilitates the identification of appropriate design patterns, optimization of performance, and reduction of maintenance efforts, particularly in complex and scientific software systems [10].

The application of GAI has been demonstrated to substantially reduce development and rework time. This reduction is achieved through the automation of time-consuming activities, such as testing, debugging, and code generation. In addition, GAI for the analysis of large-scale datasets has been demonstrated to enhance the software quality through improved defect detection. Additionally, this technology has been shown to foster innovation by supporting the generation of novel solutions and user interface designs [16].

C. ASPECTS OF STUDY

Research on GAI in SD comprises multiple dimensions, as listed in Table 1. These elements offer insights into the current state of GAI research and its applications in this domain.

TABLE 1. SD dimension in the GAI.

Aspects of study	Definition	References
Tools	GAI enables the generation of text, code, and images from natural language prompts.	[35]
Metrics	Evaluating the characteristics of GAI–generated code is essential to assess its impact on developer productivity and software quality attributes, such as consistency and functional correctness.	[34]
Method	GAI supports SD by providing context-aware code suggestions and solution strategies that enhance development efficiency and decision-making.	[35]
Collaboration & Validation	GAI collaborates with development teams by assisting in solution generation while requiring human validation to ensure correctness, safety, and compliance.	[32]
Frameworks	These approaches aim to enhance software modeling through tools that support UML model creation and translation across modeling languages.	[43]
Technology Integration	By reducing technical complexity, GAI accelerates SD and supports organizational adaptation to emerging technologies	[35]
Output Context	Understanding prompt contexts and analyzing the properties of generated outputs are critical for accurately assessing the impact of GAI on developer productivity.	[34]
Increase of Productivity	Automation of code generation and documentation through GAI allows developers to focus on higher-value design and problem-solving activities.	[22]
Ethics and Regulations	Establishing ethical and regulatory frameworks for the use of GAI is essential to ensure transparency, mitigate bias, and promote fairness across SD applications.	[44]
Roles	Emerging roles—including code assistant, code translator, documentation generator, software test generator, modeling assistant, and code researcher—contribute to improving the efficiency, reliability, and overall effectiveness of SD processes.	[34]

III. SYSTEMATIC LITERATURE REVIEW FOR SD WITH GAI

A. METHODOLOGY

A comprehensive literature review is imperative to identify and analyze recent advances and contributions to GAI in SD. To facilitate a systematic and rigorous examination, it is essential to implement well-defined review protocols and structured methodologies. These protocols and methodologies must be employed to ensure the efficient organization, synthesis, and interpretation of the collected evidence.

The methodology adopted for this systematic literature review followed a modified version of the guidelines for systematic literature reviews in software engineering proposed by Kitchenham and Charters [45]. This approach has been extensively applied in the field of software engineering research, including software testing [46], software quality assessment [47], software defect prediction using artificial intelligence [40], application of AI in software design [41], [48], and more recently, GAI in software design [11].

The review process consists of a sequence of structured stages, as illustrated in Fig. 1, and is aligned with the PRISMA workflow, shown in Fig. 2 [49]. The methodology comprises three main phases: planning, execution, and reporting. During the planning phase, the review scope was defined, including the study period, research questions, inclusion and exclusion criteria, and the search strategy. In the execution phase, the previously defined protocol was applied to identify, screen, and select primary studies that addressed the research questions. The selected studies were systematically analyzed and synthesized during the final phase of the study.

The results of the review included statistical analyses of the publication trends, journal distributions, and quartile rankings. These findings were examined in detail to address the research questions formulated during the planning phase and provide a comprehensive understanding of the current state of research on GAI in the SDLC.

To identify GAI opportunities for SD, it is necessary to examine the following research questions, which correspond to the four aspects of this study.

RQ1: WHAT GAI METHODS HAVE BEEN USED IN THE SD?

RQ2: WHAT GAI-BASED TOOLS HAVE BEEN USED IN THE SD PROCESS?

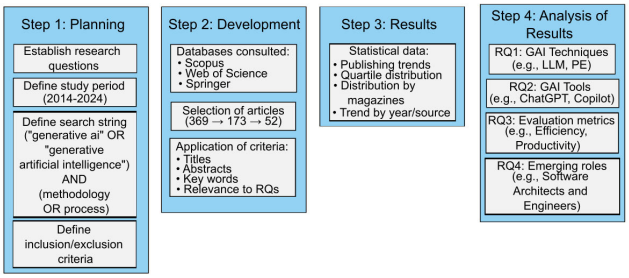


FIGURE 1. Steps of the systematic literature review process.

RQ3: WHAT METRICS ARE BEING THE UTILIZED TO ASSESS IMPACT OF GAI ON SD?
RQ4: WHAT ROLES EXIST IN SD USING GAI?

To address these questions, a structured search strategy was developed. The search strings used in this study were as follows:

("generative AI" OR "generative artificial intelligence") AND (methodology OR process).

The search string was applied to the Scopus, Web of Science (WoS), and Springer databases. In Scopus, the search was conducted using the *"Title–Abstract–Keywords"* tag, whereas in WoS the Topic tag was employed.

The selection of these databases was driven by several key factors, including broad journal coverage, consistent updates, high indexing frequency, and the availability of impact and quartile metrics. Table 2 summarizes the inclusion and exclusion criteria used in these studies.

Although research related to AI in SD appeared as early as 2014, the timeline of interest for this review was restricted to 2022–2024 to capture the recent advances specifically related to GAI. Studies focusing on adjacent topics, such as the use of natural language processing (NLP) to analyze trends in SD ([50], [51]), were excluded if they did not directly address GAI in the context of SD methodologies or processes.

Each manuscript was assigned a unique identifier and categorized according to the corresponding research question using the RXX notation. A comprehensive nomenclature and classification scheme are provided in Appendix A.A.

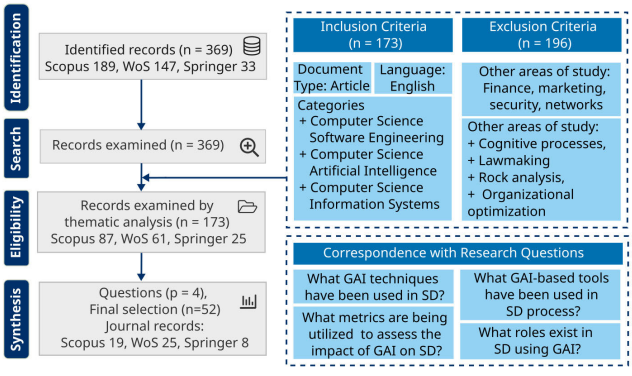


FIGURE 2. PRISMA-based source selection and screening process for the literature review.

TABLE 2. Inclusion and exclusion criteria.

Inclusion Criteria	
Type of document: article Language: English Period: 2022-2024 I should answer at least one research question. WoS Categories: ● Computer Science software Engineering ● Computer Science Artificial Intelligence ● Computer Science Information Systems	Scopus Subject Area: ● Computer Science ● Engineering Springer: ● software engineering ● Machine learning ● Programming languages, compilers, interpreters ● Programming techniques ● software engineering / programming and operating systems
Exclusion Criteria	
Another context different from SD such as marketing, image processing, finance, security, networks, IT infrastructure. Other techniques are different from GAI such as PM techniques.	

The analysis and classification of the selected studies wer conducted using LibreOffice Calc, with the studies selected for review based on defined research questions. Each primary study addressed at least one research question; however, no single study has comprehensively addressed all questions. Fig. 3 illustrates a clear and systematic overview of the thematic coverage of the reviewed literature, with the selected studies and research questions organized by the database source.

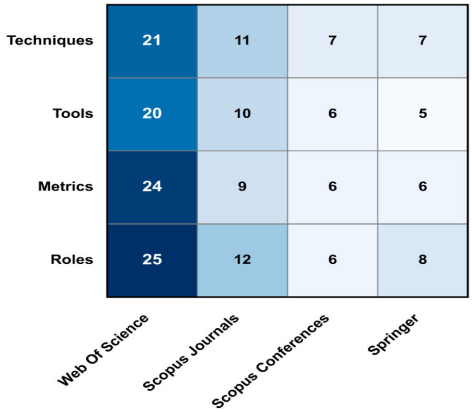


FIGURE 3. Distribution of studies according to RQs.

B. TREND ANALYSIS

The distribution of publications across databases has highlighted this growing interest. As shown in Fig. 4, the Web of Science (WoS) database accounted for the largest proportion, with 25 publications (48.07% of the total). This is followed by Scopus, which includes 19 publications, comprising journal articles, conference papers, and proceedings (36.55%), and Springer with eight publications (15.38%) (see Table 3). This distribution is indicative of two phenomena. First, there has

TABLE 3. Studies selected by research question.

RQ	Aspect	Web of Science	Scopus Journal	Scopus Conferences And Proceedings	Springer
RQ1	Methods	R1, R2, R3, R4, R5, R7, R8, R9, R11, R12, R14, R15, R16, R17, R18, R19, R20, R21, R23, R24, R25	R26, R32, R33, R35, R46, R47, R48, R49, R50, R51, R52	R28, R29, R30, R31, R34, R36, R37	R38, R39, R40, R41, R42, R43, R44
RQ2	Tools	R1, R2, R3, R4, R5, R7, R8, R10, R11, R12, R13, R14, R15, R16, R17, R18, R20, R21, R23, R24	R26, R32, R33, R35, R46, R47, R48, R49, R50, R51	R28, R29, R30, R34, R36, R37	R38, R42, R43, R44, R45
RQ3	Metrics	R1, R2, R3, R4, R5, R6, R7, R8, R10, R11, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25	R26, R27, R32, R33, R35, R46, R47, R50, R52	R29, R30, R31, R34, R36, R37	R38, R40, R41, R42, R43, R44
RQ4	Roles	R1, R2, R3, R4, R5, R6, R7, R8, R9, R10, R11, R12, R13, R14, R15, R16, R17, R18, R19, R20, R21, R22, R23, R24, R25	R26, R27, R32, R33, R35, R46, R47, R48, R49, R50, R51, R52	R28, R30, R31, R34, R36, R37	R38, R39, R40, R41, R42, R43, R44, R45
Total sources		**25**	**12**	**7**	**8**

been an increase in research activity in this area. Second, GAI in SD has been recognized as an emerging and relevant topic in software engineering.

The selected studies were classified according to the Scimago Journal Rank quartiles of the publishing venues, as shown in Fig. 5. A total of 32 journals were identified, accounting for 43 journal articles, while an additional nine publications originated from conference proceedings indexed in Scopus. Five publications appeared in the journal IEEE Access (Q1), followed by the Software and Systems Modeling Journal (Q1) with three publications. Five journals from Q2 contributed to the two publications. The category ''Other journals'' includes 25 articles that have been published across a variety of specialized venues. Publications were distributed as follows: 13 (Q1), 8 (Q2), and 4 (Q3).

The distribution of publications reflected the interdisciplinary nature of research at the intersection of GAI and SD, 90.7% (39 of 43) of journal articles were published in Q1 (23) and Q2 (16), as shown in Fig. 5. The predominance of high-impact journals reinforces the reliability of the publications' findings, thereby confirming the relevance and maturity of the literature analyzed in this study.

C. RESULTS ANALYSIS

Based on these findings, the research questions were addressed. These data were derived from the analysis of 52 primary studies.

1) RQ1: WHAT GAI METHODS HAVE BEEN USED IN SD

An analysis of 52 primary studies identified six methods of GAI applied to SD. Among the 37 studies that explicitly reported GAI methods, LLMs were the most prevalent, appearing in 20 studies. This was followed by prompt engineering (PE), which was reported in eight studies. These findings suggest that LLMs and PE are the predominant GAI methods adopted for the SD.

This widespread adoption is because of their versatility and applicability across multiple domains, including software engineering and education. A more detailed analysis shows that these methods are primarily used for technical development activities, such as source code generation, testing, and documentation. However, there is limited evidence has been found regarding their application to project management, assessment and control, and governance-related processes. Fig. 6 illustrates the distribution of the 32 distinct applications of the GAI method identified across the 37 studies, as detailed in Appendix A.B. Note that several studies have reported the same application.

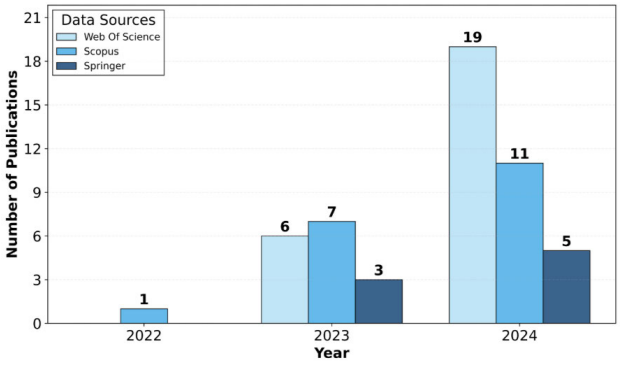


FIGURE 4. Trends in published studies by year and data source.

Methods such as retrieval-augmented generation (RAG), neural text-to-speech (NTTS), and recurrent neural networks (RNN) have received comparatively little attention in the literature. This imbalance suggests substantial opportunities for further research and adoption, particularly for exploring alternative architectures and underutilized methods. Overall, the existing body of work emphasizes mature and widely adopted solutions; however, emerging approaches remain insufficiently explored. This highlights the potential directions for methodological and architectural innovations in SD.

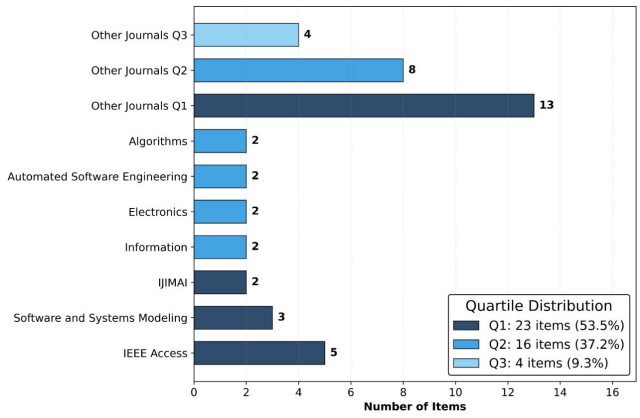


FIGURE 5. Distribution of identified studies across journal quartiles.

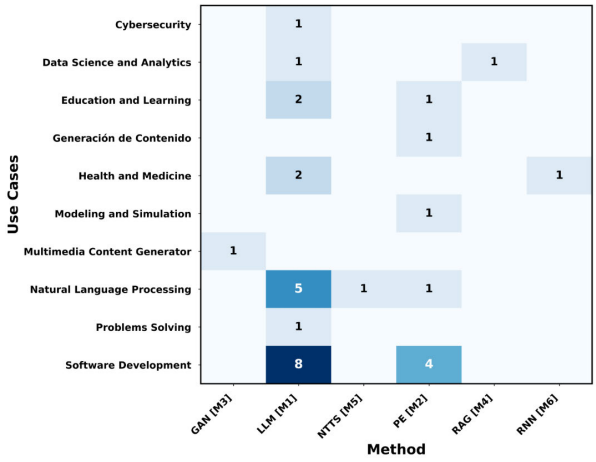


FIGURE 6. GAI methods used in SD.

2) RQ2: WHAT GAI BASED TOOLS HAVE BEEN USED IN THE SD PROCESS?

A total of 41 studies identified 32 GAI-based SD such tools. Among these, ChatGPT was the most frequently adopted tool, appearing in 21 studies (see Appendix A.C). This appendix also lists the organizations responsible for developing each identified GAI-based tool. The identified tools were then categorized by the application sector, as shown in Fig. 7.

The distribution reveals a strong trend of GAI-based tool usage in the education sector compared with other application domains. However, adoption of these tools in project management activities remains limited. Furthermore, an analysis of the SD projects revealed no evidence of GAI adoption in configuration management, quality management, or project closure activities.

Additionally, recently introduced tools, such as DeepSeek and Qwen, were not associated with any indexed publications before December 2024, the cutoff date for this review. These findings suggest that these tools have only recently emerged in research and practice, representing promising opportunities for future empirical investigation.

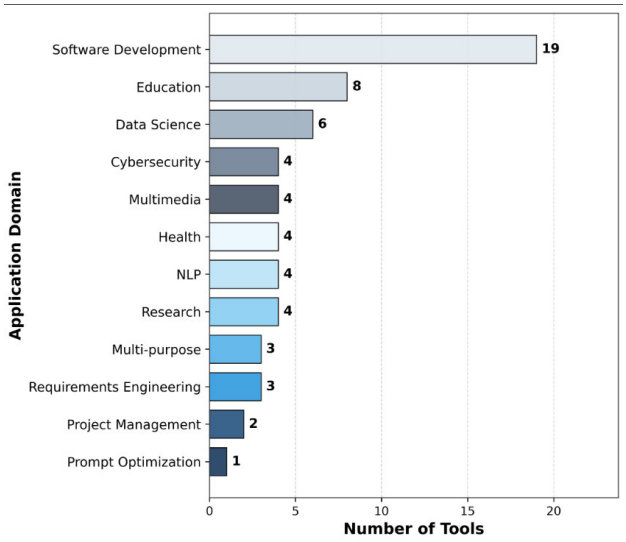


FIGURE 7. Distribution of GAI-based tools by industry sector.

3) RQ3: WHAT METRICS ARE BEING USED TO ASSESS IMPACT OF GAI ON SD?

A comprehensive evaluation framework was developed to assess the adoption and effectiveness of GAI in SD using five key characteristics. Appendix A.D provides a detailed description of the identified metrics and the guidelines for their application.

As shown in Fig. 8, the identified metrics were organized according to their inferred applicability across different phases of the SDLC. Additional consideration were given to product management and PM activities. This classification was intended to support practical adoption of the proposed metrics in specific SD contexts. The results indicate that current applications of GAI are concentrated primarily in development-related activities, while revealing emerging opportunities in the management and coordination of software projects.

The evaluation framework incorporates the following characteristics:

- Efficiency was assessed by comparing the time required to complete key SD activities across the complete SDLC, including requirements analysis [18], project planning [37], design [31], coding [22], testing [31], debugging [31], documentation [34], deployment [36], and project closure, using GAI-based tools versus conventional development approaches [51].
- Developers' productivity was evaluated based on the number of tasks completed and the relative efficiency of GAI-assisted development compared to traditional methods.
- Code quality was analyzed by examining the error rates and debugging efforts for both GAI-generated code and manually written code [51].
- Precision, a core quality attribute defined by Guide to the Software Engineering Body of Knowledge [52], must be evaluated in GAI-generated code to ensure its reliability,

consistency, and correctness, in accordance with established software engineering principles.
- User satisfaction and experience were assessed by analyzing the perceived ease of use, usefulness, and cognitive effort associated with using GAI-based tools for SD tasks [53].

Several studies have also reported that commonly used generative evaluation metrics, such as Fréchet Inception Distance (FID), Precision–Recall for Distributions (PRDC), BRISQUE, BLEU, and ROUGE, are not well-suited for evaluating GAI outputs in SD contexts. This limitation underscores the need for domain-specific evaluation metrics tailored to software engineering tasks.

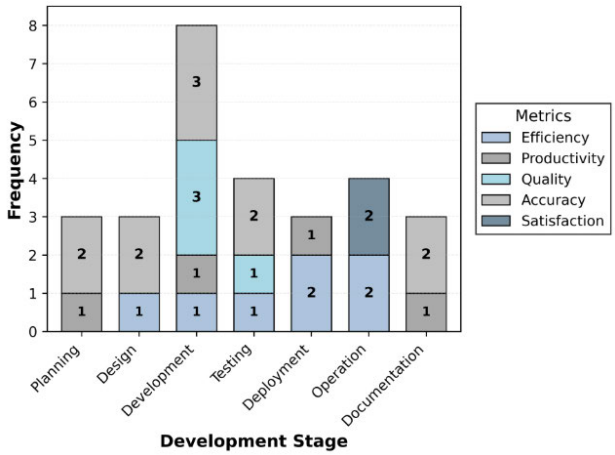


FIGURE 8. GAI metrics applied by SD stage.

4) RQ4: WHAT ROLES EXIST IN SD USING GAI?

Appendix A.E summarizes the 29 distinct SD roles identified across the 42 primary studies that investigated the application of GAI. The software developer role (P10) received the most attention, appearing in 32 studies. The machine learning engineer role (P05) appeared in 25 studies, making it the second most common role.

Fig. 9 classifies the identified roles into management and technical profiles across the SDLC. The results suggest that GAI primarily enhances technical roles and provides evidence of its impact on PM and coordination.

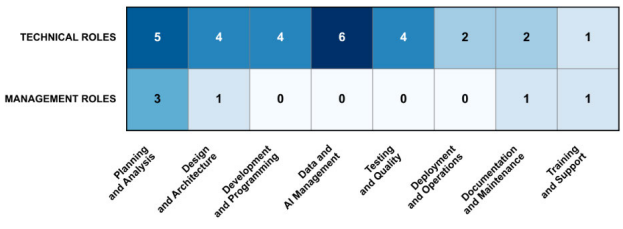


FIGURE 9. Classification of GAI-related roles in the SDLC.

IV. OPPORTUNITIES FOR THE USE OF GAI IN SD

The primary objective of this study was to identify opportunities for the effective application of GAI to SD. The proposed methodological framework supports practitioners by helping them to focus on their efforts systematically, thereby optimizing the use of time, resources, and development capacity. To accomplish this, this study proposed a five-stage methodology to support the identification, analysis, and exploitation of GAI-driven opportunities in SD.

A. METHOD

The first stage of the proposed methodology consists of an inventory phase, in which a comprehensive collection of GAI-based methods, tools, metrics, and roles is applied to SD. This inventory provides an overview of the current state of adoption and maturity of GAI in SD. As reported in Section III of the systematic review, this stage identified 6 methods, 32 tools, 5 metrics, and 29 SD roles supported by GAI.

Stage 2 focuses on identifying the applicable SD standards. The goal of this stage is to establish the reference requirements that GAI-driven opportunities should address within the SD processes. This study primarily targets small- and medium-sized software development enterprises, because they represent the most prevalent segment of the global software industry. According to [54], most firms in the European Union fall within this category and are actively engaged in digital transformation initiatives involving the development or adaptation of technological solutions. A similar trend was observed in Latin America. For example, approximately 85% of the software-exporting companies in Peru are micro and small enterprises [55].

Stage 3 involved aligning GAI opportunities with standardized SD processes. During this phase, we analyzed the processes and activities defined by the selected framework to determine where GAI methods, tools, metrics, and roles could be effectively integrated. The ISO/IEC 29110 Basic Profile is selected as the reference framework. This is because it provides a simplified and scalable process model that is well-suited for small teams and low-complexity projects. This profile includes key elements of software projects and process management and is in line with the ISO/IEC 12207 Basic Profile. It has demonstrated its effectiveness in boosting productivity for small- and medium-sized software development enterprises, particularly in emerging economies [54]. Furthermore, adopting this profile helps organizations achieve international certification, which is a critical factor for expanding software service exports. The specific requirements of the ISO/IEC 29110 Basic Profile considered in this study are summarized in Appendix B.A.

B. OPPORTUNITIES ISO/IEC 29110 BASIC PROFILE

In Stage 4, the proposed methodology focuses on identifying gaps and concludes that several requirements defined by the ISO/IEC 29110 Basic Profile are not currently addressed by existing GAI solutions. A structured three-step procedure was used to identify these gaps systematically. First, the processes defined in the Basic Profile were examined in relation to the identified progress of GAI across methods,

TABLE 4. Opportunities for using GAI for SD in ISO/IEC 29110 basic profile.

Process Group	Process	Opportunities
Project Management	Project Plan Execution	GAI enables flexible scheduling, automates role-based tasks, and accelerates the generation of customized management reports. Automate the drafting of project plans, risk matrices, and interactive dashboards using tools such as ChatGPT, Gemini, Claude, LangChain, and GitHub Copilot. By analyzing historical patterns, it is possible to determine the likelihood of deviations. This supports improved requirements traceability and enhanced decision-making for action selection. Furthermore, GAI can recommend contextualized options through automated suggestions.
	Project Assessment and Control	GAI enables the automation of project evaluation and control, as well as continuous monitoring of execution. This includes the generation of comparative reports contrasting planned versus executed activities, allowing performance analysis to be conducted more quickly and accurately than manual approaches. Tools such as ChatGPT, Gemini, and LangChain support the automated construction of cost matrices, facilitating trend visualization and early deviation detection. Project managers can make more effective real-time adjustments based on recommendations derived from historical and current project data.
	Project Closure	GAI enables the generation of checklists, closing reports, and executive summaries tailored to different target audiences, as well as structured feedback for clients and teams. Tools such as ChatGPT, Whisper, and Gemini improve the productivity and consistency of these documents. Organizational learning can be accelerated through automated tagging and organization of innovative ideas, lessons learned, and reusable patterns. Reusing this knowledge in future projects improves efficiency, strengthens decision-making, and increases software process maturity.
Software Implementation	Software Implementation Initiation	GAI supports the automation of technical plan reviews, checklist generation, and validation of pre-development conditions. GAI tools can automatically generate role assignment matrices, verify resource availability, and document initial guiding principles.
	Product Assembly	GAI facilitates the automated generation of customized, accessible, and multi-format artifacts, including technical documentation, installation scripts, user manuals, and interactive training materials. It supports the creation of acceptance reports and the validation of deliverables, ensuring effective communication, improved user experience, and a smooth transition to system operation.

tools, metrics, and roles. Second, evidence from the inventory phase (Stage 1) was analyzed for each standard process to determine whether GAI-supported practices addressed the corresponding requirements. When an SD process or activity was supported by at least one GAI aspect, it was marked Inventory (I), indicating coverage. These instances are documented in Appendix B.A. Third, the processes and activities for which no GAI support was identified were classified as gaps and labeled ¨G¨. These gaps are highlighted in blue to distinguish them from the covered requirements.

Stage 5 is the final stage of the methodology and addresses opportunity identification. During this stage, we analyzed GAI's potential of GAI to bridge the gaps identified in Stage 4. For each uncovered requirement, the analysis identified concrete opportunities to leverage the GAI and enhance SD practices. These opportunities were evaluated in terms of their feasibility and mapped to specific GAI methods (Appendix A.B), tools (Appendix A.C), metrics (Appendix A.D), and roles (Appendix A.E). Table 4 summarizes the identified process gaps and the corresponding GAI-driven opportunities proposed to address them.

These opportunities are supported by existing empirical evidence that demonstrate the successful application of GAI in comparable SD activities. For example, GAI has been shown to effectively support automatic generation of technical documentation. In the context of the Project Execution process, prior work [43] demonstrated the generation of a PlantUML code using GAI systems. This resulted in valid UML artifacts, illustrating the potential of the GAI to address the previously uncovered requirements of the ISO/IEC 29110 Basic Profile.

V. DISCUSSION

This section summarizes the findings of 52 primary studies (see Appendix A.A) to answer the four research questions and identify opportunities to close the gaps in the processes defined by the ISO/IEC 29110 Basic Profile.

A. RQ1: WHAT GAI METHODS HAVE BEEN USED IN SD?

LLMs are the most frequently used method, as reported in previous studies [31], [56], and [57]. Specifically, [56] demonstrated that LLMs can solve complex geometric problems by automatically generating executable programs. In [57], LLMs were applied to code generation tasks in Python and Java. In contrast, [31] explored their use in software testing and code refactoring.

In contrast, prompt engineering (PE) has been used in studies such as [20] and [58] to improve the quality, consistency, and contextual relevance of GAI outputs. In [20], structured prompts were designed to support requirement elicitation, whereas [58] introduced configurable prompt templates to customize generated code for non-expert users. These findings highlight PE as a complementary method to enhance the effectiveness and reliability of LLM-based solutions.

Beyond LLMs and PE, other GAI methods present clear opportunities to address the gaps in the key SD processes defined by the ISO/IEC 29110 Basic Profile. LLMs can automate activities such as generating schedules, creating technical documentation, writing installation scripts, and producing user manuals. These activities are essential for project and product deliveries. Furthermore, fine-tuning strategies improve the output quality by optimizing the instruction sets

for generating artifacts, such as checklists, role matrices, and project control reports. This supports the project monitoring and evaluation.

Generative adversarial networks (GANs) can generate visual artifacts, including software architecture diagrams and UML models, to streamline requirement analysis and technical documentation. RAG further enhances reporting capabilities by linking historical and contextual project data and; facilitating informed project evaluation and closure. In addition, NTTS methods enable the creation of audio-based training materials, thereby improving knowledge transfer and user experience during product deployment.

Overall, integrating GAI methods—namely LLMs, PE, GANs, RAG, and NTTS—across standardized SDLC phases enhances traceability, quality, and efficiency, while reducing manual effort and error rates. Nevertheless, as emphasized in [19], ethical considerations and continuous human supervision are essential to ensure responsible and trustworthy application of GAI in SD.

B. RQ2: WHAT GAI BASED TOOLS HAS BEEN USED IN THE SD PROCESS?

A detailed analysis of the reviewed studies revealed a substantial increase in the number and diversity of GAI-based SD tools. These tools exhibit varying degrees of specialization and functional focus. ChatGPT was the most cited tool, with 21 mentions of usage across the analyzed literature, including studies such as [57] and [59]. These studies employed ChatGPT for tasks such as code generation, technical documentation, and test execution support. The GitHub Copilot, discussed in [60], is an intelligent coding assistant that provides real-time code suggestions to improve developer productivity during implementation.

Other tools demonstrate more specialized capabilities. LangChain [61] facilitated the orchestration of language models and data flows, enabling the construction of complex, reasoning-based workflows and supporting chain-of-thought implementation. Whisper [62] supported the generation of meeting minutes and training documentation through automatic speech-to-text transcription. Additionally, tools such as Gemini and Auto-GPT offer advanced analytical capabilities, including autonomous task execution, multi-step reasoning, and decision -making. Domain-specific solutions, such as PentestGPT for cybersecurity applications [51] and Synthesia, focus on the generation of interactive audiovisual content for product demonstration and user training.

The diversity of these tools reflects the rapid expansion of the GAI ecosystem and its progressive integration into SDLC at various stages, including planning, implementation, delivery, and maintenance. The most widely adopted GAI tools present significant opportunities for mitigating the gaps identified in the ISO/IEC 29110 Basic Profile, particularly in project evaluation, control, and closure processes. For example, the ChatGPT can automatically generate schedules, meeting minutes, progress reports, and technical documentation. This enhances traceability, consolidates deliverables, and facilitates formal project closure. The GitHub Copilot supports code generation and validation during implementation, and its outputs also contribute to the preparation of the final deliverables. LangChain integrates multiple models and data sources to automate complex workflows, such as generating executive reports and control matrices. These reports are critical for monitoring the project objectives and performance.

Despite these advantages, the adoption of GAI for SD is accompanied by several technical, ethical, and social challenges. From a technical perspective, the generated code may be incompatible, difficult to debug, or unsafe, particularly when the quality is insufficient. Consequently, rigorous quality control and continuous human oversight are required throughout the development process [55]. Furthermore, GAI systems may produce outputs that appear syntactically and semantically correct yet are factually or logically incorrect, posing significant risks in programming and system design. Although human intervention remains essential because of current technical limitations, usability challenges arise from the unpredictability of outputs and complexity of model fine-tuning [3].

Recent studies have highlighted ethical and social concerns. A 2024 study identified privacy, security, and potential job displacement as the major issues [63]. Additional challenges affect freelance and independent developers, who must contend with the uncertainty and limitations imposed by evolving GAI frameworks [64]. In the long term, excessive reliance on GAI tools may lead to dependency and gradual erosion of developers' technical expertise, as noted in [63]. Addressing these challenges is critical for ensuring responsible, effective, and sustainable integration of GAI tools within standardized SD processes.

C. RQ3: WHAT METRICS ARE BEING USED TO ASSESS GAI IMPACT ON SD?

The literature review identified five key metrics that are commonly used to evaluate the impact of GAI on SD. The most frequently cited metric is efficiency [44], [51], and [65]. Efficiency measures the time required to complete activities, such as code generation, modeling, and documentation when using GAI versus traditional development approaches.

Another widely used metric is developer productivity, which was analyzed in [18] and [57]. This metric evaluates both the volume and quality of work completed with the assistance of GAI. These studies demonstrate that GAI-supported automation and technical text generation significantly reduce the development time while enhancing output consistency. Code quality was examined in [66] and [67], where it was assessed based on criteria such as the error rates, structural simplicity, and maintainability of the generated code.

Several studies, including [68] and [69], have examined accuracy as a key metric that reflects the reliability and consistency of the GAI outputs. High accuracy directly

TABLE 5. Sources (R) of the systematic literature review.

ID	REFERENCE	ID	REFERENCE
R01	Bartlett & Camba (2024) [79]	R27	Camara et al. (2024) [91]
R02	Alavi et al. (2024) [51]	R28	Vogelsang (2023) [18]
R03	Parra et al. (2024) [56]	R29	Deshpande & Magerko (2024) [92]
R04	Franceschelli & Musolesi (2024) [30]	R30	Acher (2024) [58]
R05	Hilario et al. (2024) [68]	R31	Parida et al., 2023 [72]
R06	Minutti-Martine et al. (2023) [80]	R32	Kim (2024) [93]
R07	Ilieva et al.(2023) [44]	R33	Bahi et al. (2024) [94]
R08	Semeler et al. (2024) [57]	R34	Weisz et al. (2022) [32]
R09	Gupta & Goyal (2024) [73]	R35	Jeong (2023) [21]
R10	Zhecheva (2024) [66]	R36	Fan et al. (2023) [31]
R11	Bukar et al. (2024) [81]	R37	Clariso & Cabot (2023) [35]
R12	Troy et al. (2023) [82]	R38	Bendel & Jabou (2024) [70]
R13	Wölfel et al. (2024) [62]	R39	Braun & Möller (2024) [95]
R14	Wang et al., 2023 [83]	R40	Combemale et al. (2023) [65]
R15	Morales-Chan et al. (2024) [61]	R41	Salvia et al. (2024) [96]
R16	Panagoulias et al. (2024) [38]	R42	Krouwel et al. (2024) [33]
R17	Loeffler et al. (2024) [84]	R43	Sauvola et al. (2024) [19]
R18	Kopitar et al. (2024) [85]	R44	Cámara et al. (2023) [43]
R19	Sobhanmanesh et al. (2023) [86]	R45	Wang & Li (2023) [97]
R20	Kim et al. (2024) [87]	R46	Llerena-Izquierdo et al. (2024) [71]
R21	Idrisov & Schlippe (2024) [60]	R47	Godwin et al. (2024) [98]
R22	Lan et al. (2024) [88]	R48	Le & Andrzejak (2024) [99]
R23	Haindl & Weinberger (2024) [89]	R49	Fitriawijaya & Jeng (2024) [100]
R24	Ranade et al. (2023) [67]	R50	Russo (2024) [59]
R25	Karpurapu et al. (2024) [90]	R51	Bedington et al. (2024) [101]
R26	Alshareef et al. (2023) [20]	R52	Diebold (2024) [69]

contributes to improved productivity, collaboration, and overall trustworthiness of the development process. Finally, user satisfaction was evaluated through surveys and direct feedback, as discussed in [70] and [71], capturing the perceptions of developers and stakeholders regarding the usability, usefulness, and cognitive effort involved in interacting with GAI-based solutions. Collectively, these metrics demonstrate the practical value and performance relevance of the GAI in real-world SD contexts.

From a standard perspective, these indicators provide concrete opportunities to address the shortcomings of the monitoring, control, and continuous improvement processes defined in the ISO/IEC 29110 Basic Profile.

Efficiency supports schedule control by quantifying the time required for technical tasks, such as coding, testing, and deployment. This reduces delays in the project execution and product delivery.

Developer productivity improves planning and execution by measuring the extent of task automation and quality of outputs produced with GAI support.

TABLE 6. GAI methods used in SD.

METHOD (M)	USAGE (U)	DESCRIPTION	SOURCE (R)
LLM [M1]	U01	Solving geometric problems	R03
	U02	Command generation, results analysis, and pretesting support	R05
	U03	Educational chatbots in education.	R07, R13
	U04	Generating scripts in Java, Python, C++ or database query language.	R08, R21, R34, R48, R50, R33
	U05	Providing feedback in courses based on rubrics defined by teachers.	R15
	U06	Supporting primary diagnostic decision-making by analyzing symptoms, recommending tests, and suggesting referrals to medical specialists.	R16
	U07	Improving the accuracy of type 2 diabetes diagnosis through AI-assisted analysis.	R18
	U08	Supporting the learning of Java concepts by suggesting algorithms, generating code, and improving existing implementations.	R23
	U09	Automatic generation of software test cases.	R25, R50
	U10	Automation of specification and validation tasks in requirements engineering.	R28
	U11	Generation UML artifacts for SD.	R27, R44
	U12	Customizing code by the creation of configurable tools and automated software variants for non-technical users.	R30
	U13	Enhancing text generation, translation, and instruction comprehension in NLP tasks.	R31
	U14	Supporting code generation, automated testing, bug fixing, refactoring, software analysis, documentation, and software engineering education.	R36
	U15	Development of chatbots with improved response consistency through prompt-based examples and validation with native speakers.	R38
	U16	Generating, debugging, and explaining code; summarizing texts; analyzing data; and creating APIs.	R46, R43
	U17	Generating, debugging, and explaining code; summarizing texts; analyzing data; and creating	R47
	U28	Automation and support exploratory data analysis.	R10
	U29	Automatic translation of virtual meetings.	R45
	U30	Writing summaries of research articles and text for social media posts.	R51
PE [M2]	U18	Generation and optimization of Discrete Event System Specification (DEVS) models, including activity description and requirements extraction.	R26
	U19	Improving output accuracy through the application of requirements engineering techniques.	R28
	U20	Supporting task standardization and the design of domain-specific user interactions.	R29
	U21	Supporting requirements generation, effort estimation, task automation,	R33

TABLE 6. *(Continued.)* GAI methods used in SD.

	U22	decision-making, and PM activities Development of a PE model based on rhetorical situations.	R24
	U23	Generation of configurable systems by adapting instructions to improve the accuracy and consistency of generated code.	R30
	U24	Creation, optimization and adaptation of prompts in a modular and platform-independent way, improving their effectiveness.	R37
	U25	Drafting proposals and personalizing texts with user data.	R48
GAN [M3]	U26	Creating images from text.	R32, R20, R49, R37
RAG [M4]	U27	Extracting relevant information from vector databases and integrating it with large language models to produce accurate responses, reducing errors and improving precision.	R35
NTTS [M5]	U31	Voice-to-text conversion.	R38
RNN [M6]	U32	Generation of structured sequences from textual descriptions to molecular structures.	R17

Software quality metrics such as consistency, clarity, and maintainability enable early identification and correction of defects during pre-delivery verification activities.

Accuracy metrics support the validation and review requirements by ensuring the reliability of the generated artifacts. User satisfaction metrics, in turn, play a critical role in confirming successful project closure and effective product transfer by assessing end-user experience.

Together, these measures provide a comprehensive overview of the SDLC, enabling organizations to track progress, evaluate effectiveness, and improve development practices continuously. Nevertheless, the integrity and transparency of GAI models are essential for mitigating risks related to security vulnerabilities and algorithmic bias. Ethical considerations are essential for ensuring the sustainable, secure, and responsible use of AI technologies in SD [3].

D. RQ4: WHAT ROLES EXIST IN SD USING GAI?

According to the literature, there are 29 roles involved or directly affected by the adoption of GAI in SD. The software developer role was the frequently addressed and cited in 32 primary studies (e.g., [43], [62] and [66]). These studies applied GAI to activities such as coding, modeling, testing, and maintenance, to improve development efficiency and software quality.

The second most prominent role is that of a machine learning engineer ([56], [72]), who is responsible for training, fine-tuning, and evaluating generative models in accordance with project-specific requirements. The roles of data scientists are closely related; their responsibilities include data preparation, analysis, and model input management, as discussed in [31] and [57]. Prompt engineering (PE) has also emerged as playing an increasingly important role

TABLE 7. GAI tools for SD.

TOOL (H)	DEVELOPER	USAGE (U)
ChatGPT [H01]	OpenAI	U01, U02, U04, U03, U05, U06, U18, U08, U22, U11, U19, U23, U17, U09, U28, U21, U27, U38, U16, U29, U30
Bing Chat [H02]	Microsoft	U01, U04
BERT [H03]	Google	U06
Google Bard [H04]	Gemini, formerly called Bard	U01, U03
Shell GPT [H05]	Farkhod Sadykov	U02
PentestGPT [H06]	Gelei Deng	U02
Auto-GPT [H07]	AutoGPT	U02
Transcoder [H08]	Meta	U04
PrivateGPT [H09]	Zylon	U05
Text-to-Speech [H10]	SlowSoft	U31
OpenAI Codex [H11]	OpenAI	U04
Open AI Api [H12]	OpenAI	U28
Perplexity AI [H13]	Perplexity AI, Inc.	U04
aiXcoder [H14]	aiXcoder Co., Ltd ()	U04
Bito AI [H15]	Bito Technologies Inc	U04
Alpaca [H16]	Stanford University	U03, U06
Vicuna [H17]	UC Berkeley	U03
Llama [H18]	Meta	U06, U04, U13, U27
Edge Chat [H19]	Microsoft	U03
Panda AI [H20]	Sinaptik GmbH	U28
Whisper [H21]	Open AI	U29
LangChain [H22]	LangChain	U03, U15, U04
REINVENT 4 [H23]	Molecular AI	U32
DALL·E [H24]	Open AI	U26
Stable Diffusion [H25]	Stability AI	U26
MidJourney [H26]	MidJourney Inc	U26
Copilot [H27]	Microsoft	U04, U08, U21
Amazon CodeWhisperer [H28]	Amazon	U04
ChatDev [H29]	OpenBMB	U16
Gemini [H30]	Google	U16
YouChat [H31]	YouChat	U30
QuillBot [H32]	QuillBot	U30

([35], [67]), emphasizing the need to design precise and context-aware instructions to elicit accurate and useful outputs from GAI systems. This role requires a combination of linguistic proficiency and technical expertise to ensure effective human–AI interaction.

In addition to these core technical roles, several supporting roles have been identified, including technical documentarians [73], educators [43], and project managers [19]. These roles benefit from the GAI-enabled automation of tasks, such as generating documentation, developing educational content, planning, monitoring, and validating software. The presence of strategic planning roles was also observed, indicating that GAI adoption influences not only operational activities, but also higher-level organizational decision-making. Overall, the findings demonstrate that GAI reshapes both the functional responsibilities and the collaborative dynamics of SD teams.

From a standard perspective, GAI presents a significant opportunity to address the functional and operational gaps in the processes defined by the ISO/IEC 29110 Basic Profile.

TABLE 8. Metrics to assess the impact of GAI on SD.

ID	METRIC	DESCRIPTION	SOURCE
E1	Efficiency	Development, deployment, execution, documentation, or testing time.	R02, R07, R17, R21, R24, R29, R33, R36 R40, R43, R50
		Number of tasks successfully automated.	R05, R07, R36, R43
		Computational/operational cost.	R06, R10, R14, R21, R24, R43
		Number of errors or bugs.	R07, R29
		Amount of required manual intervention.	R07, R43
		Support for vulnerability detection activities.	R36
		Support for requirements capture.	R40
		Support for architectural design activities.	R49
		Support for continuous deployment processes.	R43
E2	Developer productivity	Reduction in development, deployment, execution, documentation, or testing time.	R02, R08, R10, R28, R33, R43, R50, R52
		Translation of scripts from one language to another.	R34
		Automated or assisted document writing.	R47
		Support for requirements planning, tasks or backlog management.	R52
E3	Code quality	Error correction or debugging of programming code or tests.	R02, R08, R10, R01, R25, R24, R33, R43, R50
		Clarity and coherence of generated outputs, including responses, text, code, and images.	R03, R10, R16, R24, R28, R27, R37
		Error correction and code debugging of database queries	R12
		Quality of feedback provided in educational or training contexts.	R15
		Cyclomatic complexity	R01
		Code maintainability.	R01
		Generation of UML models from source code.	R27
		Correction and improvement of code generated after translation from another programming language.	R34
E4	Precision	Software testing and code bug detection accuracy.	R04, R25, R52
		Vulnerability detection in penetration testing activities.	R05
		Accuracy in medical image classification.	R06
		Precision in documentation generation.	R09, R44
		Accuracy of database query generation.	R12
		Correctness of programming code generation.	R14, R52
		SD effort estimation.	R33, R36
		Precision in architectural image generation.	R49
		Accuracy in diagnostic task.	R16
		Grammatical correctness in language translation.	R38
E5	User satisfaction and user experience	Surveys measuring developer and end-user satisfaction.	R02, R16, R36, R38, R46, R52, R43
		Perceived clarity and relevance of generated responses.	R13, R16, R46
		System response latency.	R14, R15, R20, R39
		Learning-related productivity improvements.	R51

The GAI enables a more balanced distribution of responsibilities and increased specialization within development teams, supporting compliance with standard requirements across the

TABLE 9. Roles in SD using GAI.

ID	ROLE	DESCRIPTION	SOURCE (R)
P01	Product Designers	Create objects for mass production, focusing on functionality, aesthetics and usability (R01).	R01
P02	3D Modeling Developers	Create three-dimensional models from AI-generated images, adjusting shape and texture (R01).	R01
P03	Software Engineers	Design, develop, and maintain software systems by applying established software engineering principles (R34).	R01, R03, R06, R31, R32, R34, R37, R39
P04	Data Recorders	Label and classify information to train AI models (R01).	R01
P05	Machine-Learning Engineers	Develop and optimize AI models by training them on data to generate accurate predictions (R03).	R03, R05, R06, R08, R09, R10, R11, R13, R15, R16, R17, R18, R20, R22, R23, R26, R27, R28, R31, R32, R34, R35, R39, R41, R49
P06	Educators	Guide learning processes, foster critical thinking, and facilitate the understanding of complex concepts (R03).	R03, R44, R11, R15, R23, R27, R36, R46
P07	Data Analysts	Manage, organize, and analyze digital data to support efficient access and informed decision-making (R08).	R03, R05, R06, R10, R41, R50
P08	UX Designer	Analyze how the presentation of system responses influences user experience and interaction quality (R03).	R03, R06, R16, R32, R36, R47, R50, R52
P09	Security Specialists	Identify security vulnerabilities, mitigate risks, and protect systems against cyberattacks (R05).	R05, R12
P10	Software Developers	Create and test software code, leveraging AI to improve adaptability and robustness y (R10).	R05, R08, R09, R10, R12, R13, R16, R17, R18, R20, R21, R22, R23, R25, R24, R26, R27, R28, R30, R33, R35, R36, R38, R41, R43, R44, R46, R47, R48, R49, R50, R52
P11	System Administrators	Configure development environments, manage controllers, and ensure script execution (R08).	R05, R08, R43
P12	Data Scientist	Analyze data, develop and evaluate models, tune hyperparameters, and mitigate bias to improve accuracy and explainability (R06).	R06, R08, R09, R10, R12, R13, R17, R18, R20, R22, R31, R36, R37, R39, R41
P13	Validation Specialists	Evaluate the performance of generative models, validate outputs, and ensure accuracy and reliability (R06).	R06, R09, R12, R13, R16, R17, R20, R24, R26, R27, R28, R31, R41, R44, R47
P14	Data Engineer	Design workflows, manage data files, and ensure correct data collection and processing (R08).	R08, R09, R12, R15, R16, R32, R35, R37, R49, R50
P15	Tester	Verify the functionality, quality and Verify software functionality, quality, and performance through systematic testing activities (R08).	R08, R11, R21, R25, R33, R36, R37, R38, R48, R50, R52

TABLE 9. *(Continued.)* **Roles in SD using GAI.**

P16	Specialists in Natural Language Processing	Develop systems capable of understanding, processing, and generating natural language using AI and machine learning techniques (R13).	R13, R16, R24, R27, R28, R48
P17	Software Architects	Design the structure, components and interaction of software systems and supporting technologies (R13).	R13, R32, R34, R49
P18	Project Manager	Coordinate resources, schedules, and tasks to achieve project objectives, leading software development teams (R15).	R15, R18, R33
P19	Ingenieros de DevOps	Integrate development and operations practices by automating processes to improve delivery, monitoring, and lifecycle efficiency (R35).	R35, R38, R43
P20	Technical Documentarians	Produce manuals, standards, guides, and technical documentation to support software use and maintenance (R17).	R24, R11, R17
P21	Prompt Engineers	Design and refine prompts to optimize interactions with AI models, such as LLMs, ensuring accurate, relevant, and useful outputs (R21).	R21, R10, R21, R24, R37
P22	Specialists in process modeling & simulation	They analyze, represent and optimize Analyze, model, and optimize complex processes using simulations to improve system efficiency (R26).	R26
P23	Requirements Engineer	Define, analyze, document, and manage system requirements to ensure that it meets user needs (R28).	R28
P24	Software Analyst	Define, analyze, document, and manage system requirements to ensure alignment with user and stakeholder needs (R28).	R28
P25	End Users	End users without technical expertise who customize software through simple and intuitive tools (R30).	R33
P26	Scrum Master	Guide agile teams, facilitate processes, and remove impediments to achieve project goals (R33).	R33, R52
P27	Product Owners	Manage the backlog, define requirements and prioritize tasks to maximize product value (R33).	R33, R52
P28	DSL Language Researchers and Developers	Design and develop domain-specific languages (DSL) tailored to solving problems within specific application domains (R37).	R37
P29	Blockchain Specialist	Design, implement, and maintain smart contracts and decentralized applications on blockchain platforms (R49).	R49

SDLC. For example, software developers can use tools, such as GitHub Copilot and ChatGPT, to generate and validate code, automate repetitive tasks, and improve product quality during the implementation and delivery phases. Machine learning engineers and data scientists configure, train, and

TABLE 10. **Inventory (I) and gaps (G) of GAI for SD according to the ISO/IEC 29110 basic profile.**

ISO/IEC 29110 BASIC PROFILE		I/G	Aspects of GAI in SD			
PG*	Process		Methods (M)	Tools (T)	Metrics (E)	Roles (P)
Project management	Project Planning	I	M2	H01, H27	E1, E2, E5	P18, P26, P27, P10, P15
	Project Plan Execution	G				
	Project Assessment and Control	G				
	Project Closure	G				
Software Implementation	Software Implementation Initiation	G				
	Software Requirements Analysis	I	M1, M2	H01, H27, H03, H07, H29	E3, E4	P03, P21, P03, P10, P20, P25
	Software Architectural and Detailed Design	I	M1, M2	H01, H27, H03, H06, H07, H29	E3, E1, E4, E2	P08, P10, P11, P15, P17, P21, P22
	Software Construction	I	M1, M2	H01, H27, H03, H07, H29	E3, E1, E4, E2	P03, P21, P03, P10, P20, P25
	Software Integration and Testing	I	M1, M2	H01, H27, H03, H07, H29	E1, E2, E3, E4, E5	P03, P21, P03, P10, P20, P25, P06
	Software Product Assembly	G				

contextualize generative models, thereby bridging the gaps in technical planning and specialized resource management. Prompt engineers optimize model interactions and improve output quality and support activities, such as requirements analysis and technical documentation.

Project managers benefit from GAI-assisted schedule generation, progress monitoring, and executive reporting, which enhance the project execution, control, and closure processes. Similarly, technical documentarians and educators use GAI to produce manuals, training materials, and interactive documentation, thereby facilitating knowledge transfers and system adoption. When applied across all phases of the SDLC, the GAI meets the requirements outlined in the ISO/IEC 29110 Basic Profile.

Despite its benefits, the adoption of GAI in SD raises concerns regarding accuracy, ethics, and governance. These concerns include job displacement ([74], [75], [76]), algorithmic bias, limited transparency, and privacy risks [55]. Dependence on GAI can also negatively impact team collaboration and decision-making process ([77], [78]), underscoring the importance of responsible and sustainable integration.

E. BRIEF APPLICATION EXAMPLE

For small and medium-sized enterprises in the financial software industry, manually creating SQL migration scripts between legacy and modern platforms is a recurring operational challenge. To address this issue, the team implemented a RAG system (M4) built on a fine-tuned Llama 2 (H18) LLM (M1). The RAG framework indexes the historical

TABLE 11. Opportunity 1.

Process Group	Project Management	**Process**	Project Plan Execution
Process Description			
The process involves implementing the approved project plan and agreements to deliver the software product in accordance with the defined scope, schedule, and quality objectives. This process includes assigning roles and responsibilities, allocating and managing resources, coordinating project activities, monitoring the execution progress, maintaining effective team communication, and managing changes in a controlled manner.			
Opportunities			
1	We can simplify the task assignment through PE (M2) when working with LLMs (M1). Tools such as ChatGPT (H01) and Copilot (H27) can automatically generate a list of assignments. This list is based on our main roles, for example, software Developers (P10), Product Owner (P27) and the Scrum Master (P26), with the involvement of the skills and availability of each. This method quickly and accurately spreads across multiple workers. It also minimizes delays and reduces human error. This makes the team more efficient (E1) and more productive (E2).		
2	Project manager P18 and system administrators P11 use GAI more frequently to enhance resource management. H01 ChatGPT, H22 LangChain and H20 Panda AI may assist in real-time adjustment of resources through the creation of operational plans. Teams can easily create detailed availability schedules, accurate load forecasts, and automated notifications. By utilizing AI applications, smarter changes can be made that ultimately ease the usage of the applications (E5) and enhance the economic efficiency of organizations (E1).		
3	We utilized LLMs (M1) and PE (M2) to measure the schedule variance of the project for staying on schedule. Tools such as the OpenAI API and ChatDev automatically generate smart dashboards based on the state of the project. These markings give us a summary of what has been done, but more importantly warn us early on when difficulties arise. The project manager (P18) and developers (P10) access the automated reports and initiate seasonal preparations. Using this technique, a team becomes effective (E1) and efficient (E2).		
4	GAI is important for maintaining the alignment of all stakeholders through the automation of key communication. It can draft executive reports and summarize meetings using Whisper (H21), messaging experts (P17) and end users (P25). This allows the communication of information in a more relevant way so that each group can effectively converse. This approach increases user satisfaction (E5), while reducing the frequency of misunderstandings that impose substantial costs.		
5	Project changes should be managed using (H22) LangChain with (H27) Git, GAI Copilot, and other suitable tools. This setup automatically summarizes the changes and keeps each version on the same page while keeping detailed logs of the decisions. This helps software architects (P17), managers (P18), and developers (P10) assess the impact of each change through automated simulations, thus proactively assessing risks. Rigorousness ensures that the implementation is more accurate E4 while increasing code quality E3.		

TABLE 12. Opportunity 2.

Process Group	Project Management	**Process**	Project Assessment and Control
Process Description			
The process entails the systematic observation and assessment of project progression in relation to the established Project Plan. It involves the identification of discrepancies concerning schedule, cost, and the execution of tasks. Furthermore, it encompasses the establishment and execution of corrective measures as needed. Finally, it includes the communication of project status to relevant stakeholders.			
Opportunities			
1	To facilitate a comparison between versus planned progress, the GAI through LLM (M1) and PE (M2) techniques, can automatically execute this through tools such as ChatGPT (H01), LangChain (H22) and ChatDev (H29), generating automatic reports based on Activity Logs, identifying compiled milestones, pending tasks and delays in the timeline. This improves the efficiency (E1) of Project Manager (P18) who can derive clear actionable summaries of the state of the project.		
2	To assess the compliance of tasks, costs and schedule GAI can automatically generate status matrices (for progress, costs and times) with the help of OpenAI API (H12) and Copilot (H27). These tools process internal sources (such as logs or task boards) to calculate budget compliance and deviation metrics. One result is that Manager (E2) becomes more productive. For the software architect (P17) and data scientist (P12), there is greater visibility, which allows them to interpret emerging trends and risks. To assess the compliance of tasks, costs and schedule GAI can automatically generate status matrices (for progress, costs, times) with the help of OpenAI API (H12) and Copilot (H27). These tools process internal sources (e.g., logs or task boards) to calculate budget compliance and deviation metrics. One result is that the manager (E2) becomes more productive. For the software architect (P17) and data scientist (P12), there is greater visibility, which allows them to interpret emerging trends and risks.		
3	GAI can analyze the past and present project data to detect variances and take remedial actions. Models in LangChain (such as H22) can also provide recommendations such as task reassignment or delivery postponement. The Manager (P18) takes decisions with guidance, and the teams can react quickly. It improves the accuracy (E4) and process quality (E3) by reducing negative impacts.		
4	System-generated reports to suit the audience (technical, management or client) should be generated when stakeholders need to be in the loop. To achieve this, ChatGPT (H01) or Gemini (H30) for example can be used to automatically adapt wording and style. With this, the Project Manager (P18) can maintain communication that is continuous, clear, and contextualized. This enhances user satisfaction (E5) and reinforces transparency in the PM.		

migration logs, schema documentation, and compliance guidelines. This enables the context-aware generation of accurate, transaction-safe SQL scripts with contextual awareness. A data engineer (P14) designed the retrieval pipeline, and a prompt engineer (P21) designed structured prompts to ensure regulatory compliance and referential integrity.

Implementing this system significantly reduced script development time, improved the accuracy of the generated scripts, and decreased post-migration data incidents. The project manager reported an enhanced traceability and readiness for audit compliance. This example shows how a small- and medium-sized development software enterprise can use RAG-enhanced LLMs to automate critical, domain-sensitive tasks while aligning with ISO/IEC 29110 Basic Profile processes. This improves operational efficiency and bridges standardization gaps, particularly in software implementation and product assembly.

VI. CONCLUSION

This study provides a systematic and standards-oriented analysis of how GAI can be integrated into SD processes aligned with the ISO/IEC 29110 Basic Profile. The analysis focuses on the small and medium-sized software development sector. Based on a systematic literature review of 52 primary studies published in established scientific venues, this study consolidates the existing evidence and translates it into actionable guidance for both researchers and practitioners.

First, a comprehensive and structured inventory of six GAI methods, 32 tools, five categories of evaluation metrics, and 29 SD roles was built. This inventory shows that GAI solutions are mainly used for technical development activities, such as code generation, testing, and documentation.

TABLE 13. Opportunity 3.

Process Group	Project Management	**Process**	Project Closure
Process Description			
The final process of a software project ensures an orderly and formal completion. It involves verifying that all project deliverables comply with the agreed requirements, obtaining formal customer acceptance, and completing the handover of project documentation and artifacts. The team conducts closure and retrospective meetings to evaluate project performance, identify successes and shortcomings, and document lessons learned. Closing reports are prepared to confirm that project objectives have been achieved. This process supports organizational learning and contributes to continuous improvement in future projects.			
Opportunities			
1	Project teams can implement LLM (M1) and PE (M2) to verify deliverables generated from the project. By utilizing ChatGPT (H01) and LangChain (H22), can automatically create and validate detailed checklists of the actual versus required final outputs (H66). It offers a flexible and organized method for Project Managers and Software Architects to validate and ensure compliance with the scope. Manual checks by users are a source of annoyance for vendors.		
2	At the crucial stage of customer acceptance, GAI is relieved to create key documents. Compliance reports, executive presentations, and summaries are detailed reports customized to the customer's technical level thanks to ChatGPT (H01) and Whisper (H21). Customers (P25) can modify the outputs to easily and rapidly determine the status of their products. Therefore, new services obtain easier approval and better client happiness. This document was designed for use in project closures, as directed by the project manager (P18).		
3	At retrospective meetings, GAI can be an efficient facilitator by dynamically creating an agenda, summarizing it, and perhaps performing automated project log analysis. Teams can view their earlier actions and acquire practical knowledge by leveraging OpenAI API (H12) or ChatDev (H29). Project Manager (P18) often benefited from this support. When teams grow more productive (E2) through the creation of a data-driven culture of continuous improvement, the organization's knowledge base increases.		
4	In the last stage of a project, the necessary closure artifacts were created. These include closure reports, last minutes, and a complete technical document. AI tools such as ChatGPT (H01), Bito AI (H15), and Gemini (H30) can be used to automatically generate these documents. These platforms structure, abbreviate, and adapt content to suit technical teams, management, clients, etc. Automation helps technical documentalists (P20) and project managers (P18). This reduces production times and improves quality and consistency of deliverables. As a result, the overall efficiency of the project (E1) and the resulting quality (E3) are maximized.		

TABLE 14. Opportunity 4.

Process Group	Software Implementation	**Process**	Software Implementation Initiation
Process Description			
This process marks the formal start of the software-construction phase. This process involves reviewing and validating the development plan with the project team, confirming the readiness of the technical environment, and ensuring the availability of required resources. Roles and responsibilities are clearly defined, and all necessary prerequisites are verified before development activities begin. Once these conditions are satisfied, software implementation proceeds in accordance with the specified project requirements			
Opportunities			
1	The review and validation of the development plan can be strengthened through LLMs (M1) and Prompt Engineering (M2). ChatGPT (H01) and Copilot (H27) process plans to prepare readable summaries and annotated versions, facilitating a detailed collective review by the technical team including Developers, Architects and Project Manager. We also use a platform such as LangChain, to set up collaborative sessions where AI identifies potential ambiguities or inconsistencies. This method, which superimposes a form of AI either increases efficiency (E1) and/or enhances quality (E3) from the outset of planning.		
2	GAI can automate a part of the work to ensure that the technical environment and resources are ready for project execution. The system can automatically produce resource checklists, analyze the existing environmental configuration, and recommend specific changes for optimal configurations. Platforms such as Whisper (H21) and ChatDev (H29) can automatically check infrastructure needs against project case files and technical documentation. According to DevOps Engineers (P19) and System Administrators (P11), the effective use and provision of resources helps ensure technical preparedness. This makes their teams more productive (E2).		
3	It is feasible to define team roles using prompt engineering (PE) and LLMs. Compatible applications, such as ChatGPT (H01) and the Open AI API (H12), automatically formulate a detailed responsibility assignment matrix oriented to the role, competency, and project schedule. The Project Manager (P18) can review and validate them which will then be shared with all stakeholders. When ambiguity is reduced, communication is improved. In addition, the commitment of each team member is traceable, and the team powers up (E1). Furthermore, we observed a positive impact on satisfaction (E5).		

However, PM, assessment and control, and closure activities are not well-supported.

The second contribution builds on this evidence by explicitly mapping GAI capabilities to the processes defined in the ISO/IEC 29110 Basic Profile. This mapping enabled systematic identification of unmet standard requirements, particularly within the PM and Software Implementation process groups. To address these gaps, this study proposes a set of concrete, feasible, and standards-compliant GAI–driven opportunities, as summarized in Appendix B.B, which demonstrate how GAI can be operationalized within normative process constraints rather than being applied in isolation.

The third contribution of this study was its practical feasibility. The brief application example presented in this work illustrates how RAG and LLMs can align with the Software Implementation and Product Assembly processes. This example strengthens the applicability of the proposed framework and addresses the need for empirical grounding during the review process.

Overall, the findings confirm that GAI has a significant potential to enhance efficiency, code quality, traceability, and user experience through SDLC. However, this potential is accompanied by persistent challenges related to output accuracy, hallucinations, ethical risks, limited transparency, and absence of integrated methodological guidance. Addressing these challenges requires technical solutions, process-level integration, governance mechanisms, and human supervision.

Unlike previous studies that focused on individual tools or tasks, this study systematically aligned GAI with an internationally recognized SD standard. As such, this study advances the state of the art by proposing a structured, responsible, and context-aware framework for GAI adoption, that is particularly suited to the operational constraints of small- and medium-sized software development enterprises.

Future research should extend this work through empirical validation in industrial settings, particularly within small and medium-sized software development enterprises that adopt the ISO/IEC 29110 Basic Profile. Additional efforts are required to refine domain-specific evaluation metrics,

TABLE 15. Opportunity 5.

Process Group	Software Implementation	**Process**	Software Product Assembly
Process Description			
The term "software delivery" is defined as the official conveyance of the software product to the client, in accordance with the terms of the prior agreement. Software delivery is subject to a series of conditions, including the preparation of the product, its transfer, its acceptance, the training of users, and the closure of the project.			
Opportunities			
1	When developing and preparing products, teams can use LLMs (M1) under PE (M2) to automate documentation. We leveraged tools such as ChatGPT (H01), Bito AI (H15) and LangChain (H22) to efficiently produce technical documentation, manuals, and one detailed checklist. Technical documentarians (P20) and software architects (P17) benefit enormously from this automation. The real benefits of GAI integration are apparent in terms of performance measurements, where the improved efficiency metric (E1) can be easily tracked.		
2	PE can help automate the product migration process, mostly by generating accurate installation, deployment and configuration scripts for the target environment. LangChain and Whisper can help document each step and verify the entire migration process. Efficiency metrics (E1) and customer experience metrics (E5) help us measure their success. This necessitates a reconsideration of the role of the DevOps engineer (P19), who is becoming increasingly concerned with validating and monitoring AI-produced configurations in production and less concerned with the manual design of intricate continuous deployment pipelines.		
3	We can automate product acceptance using tricks. The GAI can make compliance reports, qualification criteria, and summarize translated technical-language determined by the extent of the client's comprehension. Once this clarity is achieved, misunderstandings are diminished and feedback interactions speed up, leading to much higher satisfaction (E5). In this context, Project Managers (P18) can become more effective by leveraging AI-generated documentation to respond more accurately and efficiently to client expectations and inquiries.		
4	The GAI can be used to create materials that can help train users more efficiently. Tools such as ChatGPT can write how-to guides, Gamma.app can create presentation decks, and Synthesia can create personalized videos. This method makes it possible to create personalized audiovisuals that make learning easier thanks to the simplification of complicated subjects. User satisfaction increases as comprehension decreases (E5). At this stage, educators (P06) and technical documentarians (P20) collaborate to adapt and validate the contents as per the end users' (P25) level of knowledge and needs.		
5	Using GAI to compile final reports, minutes, and executive summaries in the project closure phase can be highly productive. ChatGPT (H01) facilitates automated production of technical documentation. This contains well-structured and coherent content that is contextualized according to the target audience, such as technical teams, management, or clients. This makes communication in the final phase more lucid and significantly affects the overall efficiency (E1).		

formalize ethical and governance mechanisms, and evaluate the long-term impact of GAI on productivity, software quality, and workforce roles. Furthermore, international software engineering standards must evolve to explicitly incorporate the GAI as a first-class element throughout the SDCL. These advances are essential to ensure responsible, scalable, and sustainable integration of GAI into standardized SD practices.

APPENDIX A
SOURCES OF THE SYSTEMATIC LITERATURE REVIEW

A. SELECTED SOURCES (R)

See Table 5.

B. GAI METHODS THAT HAVE BEEN USED IN SD

See Table 6.

C. GAI TOOLS FOR SD

See Table 7.

D. METRICS TO ASSESS THE IMPACT OF GAI ON SD

See Table 8.

E. ROLES IN SD USING GAI

See Table 9.

APPENDIX B
GAI - GAP AND OPPORTUNITY ANALYSIS

F. INVENTORY (I), AND GAPS (G) OF GAI FOR SD ACCORDING TO ISO/IEC 29110 BASIC PROFILE

In the matrix, cells highlighted in blue denote identified gaps (G), whereas cells highlighted in white denote inventory components (I). This study subsequently addressed the gaps identified. See Table 10.

G. DESCRIPTION OF GAI'S 5 OPPORTUNITIES ON THE IDENTIFIED GAPS IN ISO/IEC 29110 BASIC PROFILE

See Tables 11–15.

REFERENCES

[1] J. Pereira, J. M. López, M. Azanza, Ó. Díaz, and X. Garmendia, "Leveraging on generative AI to teach undergraduate software engineering: A collaborative prompt catalogue," in *Proc. 11th Int. Conf. Technol. Ecosyst. Enhancing Multicult. (TEEM)*, Oct. 2023, pp. 125–134, doi: 10.1007/978-981-97-1814-6_12.

[2] M. M. Nair, A. K. Tyagi, and N. Sreenath, "The future with Industry 4.0 at the core of society 5.0: Open issues, future opportunities and challenges," in *Proc. Int. Conf. Comput. Commun. Informat. (ICCCI)*, Jan. 2021, pp. 1–7, doi: 10.1109/ICCCI50826.2021.9402498.

[3] A. Ravera and C. Gena, "On the usability of generative AI: Human generative AI," in *Proc. CEUR Workshop*, vol. 3957, 2025. [Online]. Available: https://ceur-ws.org/Vol-3957/SOCIALIZE-paper11.pdf

[4] N. Rane, "ChatGPT and similar generative artificial intelligence (AI) for building and construction industry: Contribution, opportunities and challenges of large language models for Industry 4.0, Industry 5.0, and society 5.0," *SSRN Electron. J.*, 2023, doi: 10.2139/ssrn.4603221. [Online]. Available: https://ssrn.com/abstract=.4603221

[5] J. Amankwah-Amoah, S. Abdalla, E. Mogaji, A. Elbanna, and Y. K. Dwivedi, "The impending disruption of creative industries by generative AI: Opportunities, challenges, and research agenda," *Int. J. Inf. Manage.*, vol. 79, Dec. 2024, Art. no. 102759, doi: 10.1016/j.ijinfomgt.2024.102759.

[6] T. Kumar, V. Garg, S. Lalar, and R. Kumar, "Measuring impact of generative AI in software development and innovation," in *Intelligent IT Solutions for Sustainability in Industry 5.0 Paradigm. ICEIL 2023* (Lecture notes in Networks and Systems), B. Shukla, B. K. Murthy, N. Hasteer, H. Kaur, and J. P. Van Belle, Eds., Singapore: Springer, 2024, pp. 57–67, doi: 10.1007/978-981-97-1682-1_6.

[7] C. V. S. Babu and K. H. Kishore, "Generative AI for source code creation: Revolutionizing cloud-native software engineering," in *Artificial Intelligence for Cloud-Native Software Engineering*, B. Shukla, B. K. Murthy, N. Hasteer, H. Kaur, and J. P. Van Belle, Eds., Hershey, PA, USA: IGI Global, 2025, ch. 4, pp. 87–116, doi: 10.4018/979-8-3693-9356-7.ch004.

[8] J. A. Aguilar-Calderón, C. Tripp-Barba, P. A. Aguilar-Calderón, A. Zaldívar-Colado, and P. A. Aguilar-Calderón, "Generative AI in web engineering: Transforming development, design, and documentation," in *Generative AI in Software Engineering*, B. Shukla, B. K. Murthy, N. Hasteer, H. Kaur, and J. P. Van Belle, Eds., Hershey, PA, USA: IGI Global, 2025, pp. 263–286, doi: 10.4018/979-8-3373-0370-3.ch008.

[9] Y. Kumar, C. R. K. Babu, M. M. Deorari, L. Jasim, K. Sharma, and K. Vinothkumar, "An enhanced role of AI in SDLC in implementing maintenance system," in *Proc. 4th Int. Conf. Advance Comput. Innov. Technol. Eng. (ICACITE)*, Greater Noida, India, May 2024, pp. 1064–1068, doi: 10.1109/ICACITE60783.2024.10616905.

[10] E.-M. Arvanitou, A. Ampatzoglou, A. Chatzigeorgiou, and J. C. Carver, "Software engineering practices for scientific software development: A systematic mapping study," *J. Syst. Softw.*, vol. 172, Feb. 2021, Art. no. 110848, doi: 10.1016/j.jss.2020.110848.

[11] A. Nguyen-Duc, B. Cabrero-Daniel, A. Przybylek, C. Arora, D. Khanna, T. Herda, U. Rafiq, J. Melegati, E. Guerra, K.-K. Kemell, M. Saari, Z. Zhang, H. Le, T. Quan, and P. Abrahamsson, "Generative artificial intelligence for software engineering—A research agenda," 2023, *arXiv:2310.18648*.

[12] V. Shah, "Towards efficient software engineering in the era of AI and ML: Best practices and challenges," *Int. J. Comput. Sci. Technol.*, vol. 3, no. 3, pp. 63–78, 2019. [Online]. Available: https://ijcst.com.pk/index.php/IJCST/article/view/372

[13] P. Abrahamsson, N. Oza, and M. T. Siponen, "Agile software development methods: A comparative review," in *Agile Software Development*. Berlin, Germany: Springer, 2010, pp. 31–59, doi: 10.1007/978-3-642-12575-1_3.

[14] F. Alaydrus, T. Raharjo, B. Hardian, and A. Prasetyo, "Approaches in determining software development methods for organizations: A systematic literature review," in *Proc. IEEE Int. IoT, Electron. Mechatronics Conf. (IEMTRONICS)*, Apr. 2021, pp. 1–6, doi: 10.1109/IEMTRONICS52119.2021.9422662.

[15] H. Edison, X. Wang, and K. Conboy, "Comparing methods for large-scale agile software development: A systematic literature review," *IEEE Trans. Softw. Eng.*, vol. 48, no. 8, pp. 2709–2731, Aug. 2022, doi: 10.1109/TSE.2021.3069039.

[16] C. Ebert and P. Louridas, "Generative AI for software practitioners," *IEEE Softw.*, vol. 40, no. 4, pp. 30–38, Jul. 2023, doi: 10.1109/MS.2023.3265877.

[17] S. Mezzaro, A. Gambi, and G. Fraser, "An empirical study on how large language models impact software testing learning," in *Proc. 28th Int. Conf. Eval. Assessment Softw. Eng.*, Salerno, Italy, Jun. 2024, pp. 555–564, doi: 10.1145/3661167.3661273.

[18] C. Arora, J. Grundy, and M. Abdelrazek, "Advancing requirements engineering through generative AI: Assessing the role of LLMs," 2023, *arXiv:2310.13976*.

[19] J. Sauvola, S. Tarkoma, M. Klemettinen, J. Riekki, and D. Doermann, "Future of software development with generative AI," *Automated Softw. Eng.*, vol. 31, no. 1, p. 26, May 2024, doi: 10.1007/s10515-024-00426-z.

[20] A. Alshareef, N. Keller, P. Carbo, and B. P. Zeigler, "Generative AI with modeling and simulation of activity and flow-based diagrams," in *Proc. SIMUTOOLS*. Cham, Switzerland: Springer, 2024, pp. 95–109, doi: 10.1007/978-3-031-57523-5_8.

[21] C. Jeong, "A study on the implementation of generative AI services using an enterprise data-based LLM application architecture," *Adv. Artif. Intell. Mach. Learn.*, vol. 3, no. 4, pp. 1588–1618, 2023, doi: 10.54364/aaiml.2023.1191.

[22] M. Coutinho, L. Marques, A. Santos, M. Dahia, C. França, and R. de Souza Santos, "The role of generative AI in software development productivity: A pilot case study," in *Proc. 1st ACM Int. Conf. AI-Powered Softw.*, Chicago, IL, USA, Jul. 2024, pp. 131–138, doi: 10.1145/3664646.3664773.

[23] D. Al-Janabi, M. F. Enache, and T. S. Letia, "The quality of programs conceived by object enhanced time Petri nets," in *Proc. 7th Int. Conf. Control, Decis. Inf. Technol. (CoDIT)*, vol. 1, Prague, Czech Republic, Jun. 2020, pp. 768–773, doi: 10.1109/CoDIT49905.2020.9263964.

[24] A. K. Agarwal, R. G. Tiwari, V. Khullar, R. Ahuja, and T. Sharma, "Measuring the bytecode quality of object-oriented software," in *Soft Computing: Theories and Applications* (Lecture Notes in Networks and Systems). Singapore: Springer, 2022, pp. 45–52, doi: 10.1007/978-981-19-0707-4_5.

[25] F. Héjja, T. Bartók, R. Dakroub, and G. Kocsis, "Generative AI for productivity in industry and education," in *Proc. 9th Int. Conf. Complex., Future Inf. Syst. Risk*, Angers, France, 2024, pp. 128–135, doi: 10.5220/0012736200003708.

[26] M. Fischer and C. Lanquillon, "Evaluation of generative AI-assisted software design and engineering: A user-centered approach," in *Artificial Intelligence in HCI. (HCII 2024)*, H. Degen and S. Ntoa, Eds., Cham, Switzerland: Springer, 2024, pp. 31–47, doi: 10.1007/978-3-031-60606-9_3.

[27] Z. Li, X. Wang, J. Zhao, S. Yang, G. Du, X. Hu, B. Zhang, Y. Ye, Z. Li, R. Zhao, and H. Mao, "PET-SQL: A prompt-enhanced two-round refinement of text-to-SQL with cross-consistency," 2024, *arXiv:2403.09732*.

[28] H. Keuning, A. Luxton-Reilly, C. Ott, A. Petersen, and N. Kiesler, "Goodbye hello world–research questions for a future CS1 curriculum," in *Proc. 24th Koli Calling Int. Conf. Comput. Educ. Res.*, Lieksa, Finland, Nov. 2024, pp. 1–2, doi: 10.1145/3699538.3699591.

[29] Secretaría de Economía. (2005). *Modelo De Procesos Para La Industria De Software: MoProSoft (Versión 1.3)*. Gobierno de México. Accessed: Jan. 26, 2025. [Online]. Available: https://www.sige.org.mx/verificacion-moprosoft/.

[30] G. Franceschelli and M. Musolesi, "Reinforcement learning for generative AI: State of the art, opportunities and open research challenges," *J. Artif. Intell. Res.*, vol. 79, pp. 417–446, Feb. 2024, doi: 10.1613/jair.1.15278.

[31] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta, S. Yoo, and J. M. Zhang, "Large language models for software engineering: Survey and open problems," in *Proc. IEEE/ACM Int. Conf. Softw. Eng., Future Softw. Eng. (ICSE-FoSE)*, May 2023, pp. 31–53, doi: 10.1109/ICSE-FOSE59343.2023.00008.

[32] J. D. Weisz, M. Müller, S. I. Ross, F. Martinez, S. Houde, M. Agarwal, K. Talamadupula, and J. T. Richards, "Better together? An evaluation of AI-supported code translation," in *Proc. 27th Int. Conf. Intell. User Interface*, Helsinki, Finland, Mar. 2022, pp. 369–391, doi: 10.1145/3490099.3511157.

[33] M. R. Krouwel, M. Op 't Land, and H. A. Proper, "From enterprise models to low-code applications: Mapping DEMO to mendix; illustrated in the social housing domain," *Softw. Syst. Model.*, vol. 23, no. 4, pp. 837–864, Aug. 2024, doi: 10.1007/s10270-024-01156-2.

[34] J. Sun, Q. V. Liao, M. M´uller, M. Agarwal, S. Houde, K. Talamadupula, and J. D. Weisz, "Investigating explainability of generative AI for code through scenario-based design," in *Proc. 27th Int. Conf. Intell. User Interface*, Helsinki, Finland, Mar. 2022, pp. 212–228, doi: 10.1145/3490099.3511119.

[35] R. Clarisó and J. Cabot, "Model-driven prompt engineering," in *Proc. ACM/IEEE 26th Int. Conf. Model Driven Eng. Lang. Syst. (MODELS)*, Oct. 2023, pp. 47–54, doi: 10.1109/MODELS58315.2023.00020.

[36] K. Lebioda, V. Vorobev, N. Petrovic, F. Pan, V. Zolfaghari, and A. Knoll, "Towards single-system illusion in software-defined vehicles—Automated, AI-powered workflow," 2024, *arXiv:2403.14460*.

[37] R. Hoda, H. Dam, C. Tantithamthavorn, P. Thongtanunam, and M.-A. Storey, "Augmented agile: human-centered AI-assisted software management," *IEEE Softw.*, vol. 40, no. 4, pp. 106–109, Jul. 2023, doi: 10.1109/MS.2023.3268725.

[38] D. P. Panagoulias, M. Virvou, and G. A. Tsihrintzis, "Augmenting large language models with rules for enhanced domain-specific interactions: The case of medical diagnosis," *Electronics*, vol. 13, no. 2, p. 320, Jan. 2024, doi: 10.3390/electronics13020320.

[39] *Systems and Software Engineering-Life Cycle Profiles for Very Small Entities (VSEs)—Part 5-1-2: Software Engineering Guidelines for the Generic Basic Profile*, document ISO/IEC 29110-5-1-2:2025, Feb. 2025.

[40] J. Pachouly, S. Ahirrao, K. Kotecha, G. Selvachandran, and A. Abraham, "A systematic literature review on software defect prediction using artificial intelligence: Datasets, data validation methods, approaches, and tools," *Eng. Appl. Artif. Intell.*, vol. 111, May 2022, Art. no. 104773, doi: 10.1016/j.engappai.2022.104773.

[41] A. H. Mohammadkhani, N. S. Bommi, M. Daboussi, O. Sabnis, C. Tantithamthavorn, and H. Hemmati, "A systematic literature review of explainable AI for software engineering," 2023, *arXiv:2302.06065*.

[42] M. Kessel and C. Atkinson, "Morescient GAI for software engineering," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–17, May 2025, doi: 10.1145/3709354.

[43] J. Cámara, J. Troya, L. Burgueño, and A. Vallecillo, "On the assessment of generative AI in modeling tasks: An experience report with ChatGPT and UML," *Softw. Syst. Model.*, vol. 22, no. 3, pp. 781–793, Jun. 2023, doi: 10.1007/s10270-023-01105-5.

[44] G. Ilieva, T. Yankova, S. Klisarova-Belcheva, A. Dimitrov, M. Bratkov, and D. Angelov, "Effects of generative chatbots in higher education," *Information*, vol. 14, no. 9, p. 492, Sep. 2023, doi: 10.3390/info14090492.

[45] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele Univ., Durham Univ., Keele, U.K., Tech. Rep. EBSE-2007-01, 2007. [Online]. Available: https://legacyfileshare.elsevier.com/promis_misc/525444systematicreviewsguide.pdf

[46] V. Garousi and M. V. Mäntylä, "A systematic literature review of literature reviews in software testing," *Inf. Softw. Technol.*, vol. 80, pp. 195–216, Dec. 2016, doi: 10.1016/j.infsof.2016.09.002.

[47] G. Arcos-Medina and D. Mauricio, "Aspects of software quality applied to the process of agile software development: A systematic literature review," *Int. J. Syst. Assurance Eng. Manage.*, vol. 10, no. 5, pp. 867–897, Oct. 2019, doi: 10.1007/s13198-019-00840-7.

[48] T. Clement, N. Kemmerzell, M. Abdelaal, and M. Amberg, "XAIR: A systematic metareview of explainable AI (XAI) aligned to the software development process," *Mach. Learn. Knowl. Extraction*, vol. 5, no. 1, pp. 78–108, Jan. 2023, doi: 10.3390/make5010006.

[49] M. J. Page, "The PRISMA 2020 statement: An updated guideline for reporting systematic reviews," *Systematic Rev.*, vol. 10, no. 1, p. 89, Mar. 2021, doi: 10.1186/s13643-021-01626-4.

[50] A. Hindle, E. T. Barr, Z. Su, M. Gabel, and P. Devanbu, "On the naturalness of software," in *Proc. 34th Int. Conf. Softw. Eng. (ICSE)*, Zurich, Switzerland, Jun. 2012, pp. 837–847, doi: 10.1109/ICSE.2012.6227135.

[51] M. Alavi, D. E. Leidner, and R. Mousavi, "Knowledge management perspective of generative artificial intelligence," *J. Assoc. for Inf. Syst.*, vol. 25, no. 1, pp. 1–12, 2024, doi: 10.17705/1jais.00859.

[52] IEEE Computer Society. (2024). *Guide to the Software Engineering Body of Knowledge (SWEBOK)*. version 4.0. Accessed: Jan. 26, 2025. [Online]. Available: https://www.computer.org/education/bodies-of-knowledge/software-engineering

[53] J. Brodny and M. Tutak, "Digitalization of small and medium-sized enterprises and economic growth: Evidence for the EU-27 countries," *J. Open Innov., Technol., Market, Complex.*, vol. 8, no. 2, p. 67, Jun. 2022, doi: 10.3390/joitmc8020067.

[54] C. Y. Laporte, M. Muñoz, and B. Gerançon, "The education of students about ISO/IEC 29110 software engineering standards and their implementations in very small entities," in *Proc. IEEE Canada Int. Humanitarian Technol. Conf. (IHTC)*, Toronto, ON, Canada, Jul. 2017, pp. 94–98, doi: 10.1109/IHTC.2017.8058208.

[55] L. E. Popok and V. V. Mantulenko, "Problem of using generative artificial intelligence in the development of software products," in *Proc. XI Int. Sci. Conf. Digit. Transform. Econ., Challenges, Trends, New Opportunities (ISCDTE)*. Cham, Switzerland: Springer, 2024, pp. 13–24, doi: 10.1007/978-3-031-65662-0_2.

[56] V. Parra, P. Sureda, A. Corica, S. Schiaffino, and D. Godoy, "Can generative AI solve geometry problems? Strengths and weaknesses of LLMs for geometric reasoning in Spanish," *Int. J. Interact. Multimedia Artif. Intell.*, vol. 8, no. 5, pp. 65–74, Mar. 2024, doi: 10.9781/ijimai.2024.02.009.

[57] A. Semeler, A. Pinto, T. Koltay, T. Dias, A. Oliveira, J. González, and H. B. F. Rozados, "ALGORITHMIC LITERACY: Generative artificial intelligence technologies for data librarians," *ICST Trans. Scalable Inf. Syst.*, vol. 11, no. 2, pp. 1–11, Jan. 2024, doi: 10.4108/eetsis.4067.

[58] M. Acher, "A demonstration of end-user code customization using generative AI," in *Proc. 18th Int. Work. Conf. Variability Model. Softw.-Intensive Syst.*, Bern, Switzerland, Feb. 2024, pp. 139–145, doi: 10.1145/3634713.3634732.

[59] D. Russo, "Navigating the complexity of generative AI adoption in software engineering," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 5, pp. 1–50, Jun. 2024, doi: 10.1145/3652154.

[60] B. Idrisov and T. Schlippe, "Program code generation with generative AIs," *Algorithms*, vol. 17, no. 2, p. 62, Jan. 2024, doi: 10.3390/a17020062.

[61] M. Morales-Chan, H. R. Amado-Salvatierra, J. A. Medina, R. Barchino, R. Hernández-Rizzardini, and A. M. Teixeira, "Personalized feedback in massive open online courses: Harnessing the power of LangChain and OpenAI API," *Electronics*, vol. 13, no. 10, p. 1960, May 2024, doi: 10.3390/electronics13101960.

[62] M. Wölfel, M. B. Shirzad, A. Reich, and K. Anderer, "Knowledge-based and generative-AI-driven pedagogical conversational agents: A comparative study of Grice's cooperative principles and trust," *Big Data Cognit. Comput.*, vol. 8, no. 1, p. 2, Dec. 25, 2023, doi: 10.3390/bdcc8010002.

[63] T. Mbizo, G. Oosterwyk, P. Tsibolane, and P. Kautondokwa, "Cautious optimism: The influence of generative AI tools in software development projects," in *Proc. 25th Int. Conf. Bus. Inf. Syst. (BIS)*. Cham, Switzerland: Springer, 2024, pp. 361–373, doi: 10.1007/978-3-031-64881-6_21.

[64] M. Dolata, N. Lange, and G. Schwabe, "Development in times of hype: How freelancers explore generative AI?" in *Proc. IEEE/ACM 46th Int. Conf. Softw. Eng.*, Lisbon, Portugal, Apr. 2024, pp. 1–13, doi: 10.1145/3597503.3639111.

[65] B. Combemale, J. Gray, and B. Rumpe, "ChatGPT in software modeling," *Softw. Syst. Model.*, vol. 22, no. 3, pp. 777–779, Jun. 2023, doi: 10.1007/s10270-023-01106-4.

[66] D. Zhecheva, "Exploratory data analysis and the rise of large language models–gaming industry insights," *TEM J.*, vol. 1, no. 1, pp. 561–569, Feb. 2024, doi: 10.18421/tem131-59.

[67] N. Ranade, M. Saravia, and A. Johri, "Using rhetorical strategies to design prompts: A human-in-the-loop approach to make AI useful," *AI Soc.*, vol. 40, no. 2, pp. 711–732, Feb. 2025, doi: 10.1007/s00146-024-01905-3.

[68] E. Hilario, S. Azam, and J. Sundaram, "Generative AI for pentesting: The good, the bad, the ugly," *Int. J. Inf. Secur.*, vol. 23, no. 3, pp. 2075–2097, 2024, doi: 10.1007/s10207-024-00835-x.

[69] P. Diebold, "From backlogs to bots: Generative AI's impact on agile role evolution," *J. Softw., Evolution Process*, vol. 37, no. 1, Nov. 2024, Art. no. e2740, doi: 10.1002/smr.2740.

[70] O. Bendel and D. Jabou, "@llegra: A chatbot for vallader," *Int. J. Inf. Technol.*, vol. 16, no. 4, pp. 2035–2045, Apr. 2024, doi: 10.1007/s41870-024-01779-0.

[71] J. Llerena-Izquierdo, J. Mendez-Reyes, R. Ayala-Carabajo, and C. Andrade-Martinez, "Innovations in introductory programming education: The role of AI with Google colab and Gemini," *Educ. Sci.*, vol. 14, no. 12, p. 1330, Dec. 2024, doi: 10.3390/educsci14121330.

[72] S. Parida, S. Sekhar, S. Panda, S. Jena, A. Parida, S. K. Sahoo, and S. R. Dash, "Olive: An instruction following LLaMA model for odia language," in *Proc. IEEE Silchar Subsection Conf. (SILCON)*, Silchar, India, Nov. 2023, pp. 1–7, doi: 10.1109/SILCON59133.2023.10404195.

[73] A. Gupta and R. Goyal, "A generative AI-driven method-level semantic clone detection based on the structural and semantical comparison of methods," *IEEE Access*, vol. 12, pp. 70773–70791, 2024, doi: 10.1109/ACCESS.2024.3401770.

[74] V. Sharma, S. Deb, Y. Mahajan, A. Ghosal, and M. Kapse, "Psychological impacts of AI-induced job displacement among Indian IT professionals: A delphi-validated thematic analysis," *Int. J. Qualitative Stud. Health Well-Being*, vol. 20, no. 1, Dec. 2025, Art. no. 2556445, doi: 10.1080/17482631.2025.2556445.

[75] G. Mingjie, M. Jun, and T. Huirong, "The double-edged sword effect of generative artificial intelligence usage on employee creativity: A perspective of conservation of resources theory," *Current Psychol.*, vol. 44, no. 10, pp. 8485–8497, May 2025, doi: 10.1007/s12144-025-07806-y.

[76] M. Kam, C. Miller, M. Wang, A. Tidwell, I. A. Lee, J. Malyn-Smith, B. Perret, V. Tiwari, J. Kenitzer, A. Macvean, and E. Barrar, "What do professional software developers need to know to succeed in an age of artificial intelligence?" in *Proc. 33rd ACM Int. Conf. Found. Softw. Eng.*, Jun. 2025, pp. 947–958, doi: 10.1145/3696630.3727251.

[77] N. Del Sarto, E. Bocchialini, L. Gai, and F. Ielasi, "Digital banking: How social media is shaping the game," *J. Financ. Manage., Markets Inst.*, vol. 17, no. 2, pp. 348–363, Feb. 2025.

[78] S. Rico and L.-M. Öberg, "Challenges and opportunities for generative AI in software engineering: A managerial view," in *Proc. 33rd ACM Int. Conf. Found. Softw. Eng.*, Jun. 2025, pp. 1338–1344, doi: 10.1145/3696630.3728718.

[79] K. A. Bartlett and J. D. Camba, "Generative artificial intelligence in product design education: Navigating concerns of originality and ethics," *Int. J. Interact. Multimedia Artif. Intell.*, vol. 8, no. 5, pp. 55–64, Mar. 2024, doi: 10.9781/ijimai.2024.02.006.

[80] C. Minutti-Martinez, B. Escalante-Ramírez, and J. Olveres-Montiel, "PumaMedNet-CXR: An explainable generative artificial intelligence for the analysis and classification of chest X-ray images," in *Proc. Advances in Soft Comput., 22nd Mexican Int. Conf. Artif. Intell. (MICAI)*. Cham, Switzerland: Springer, 2023, pp. 211–224, doi: 10.1007/978-3-031-47640-2_18.

[81] U. A. Bukar, M. S. Sayeed, S. F. A. Razak, S. Yogarayan, and O. A. Amodu, "An integrative decision-making framework to guide policies on regulating ChatGPT usage," *PeerJ Comput. Sci.*, vol. 10, p. e1845, Feb. 2024, doi: 10.7717/peerj-cs.1845.

[82] C. Troy, S. Sturley, J. M. Alcaraz-Calero, and Q. Wang, "Enabling generative AI to produce SQL statements: A framework for the auto-generation of knowledge based on EBNF context-free grammars," *IEEE Access*, vol. 11, pp. 123543–123564, 2023, doi: 10.1109/ACCESS.2023.3329071.

[83] Y.-C. Wang, J. Xue, C. Wei, and C.-C.-J. Kuo, "An overview on generative AI at scale with edge–cloud computing," *IEEE Open J. Commun. Soc.*, vol. 4, pp. 2952–2971, 2023, doi: 10.1109/OJCOMS.2023.3320646.

[84] H. H. Loeffler, J. He, A. Tibo, J. P. Janet, A. Voronov, L. H. Mervin, and O. Engkvist, "Reinvent 4: Modern AI–driven generative molecule design," *J. Cheminformatics*, vol. 16, no. 1, p. 20, Feb. 2024, doi: 10.1186/s13321-024-00812-5.

[85] L. Kopitar, I. Fister, and G. Stiglic, "Using generative AI to improve the performance and interpretability of rule-based diagnosis of type 2 diabetes mellitus," *Information*, vol. 15, no. 3, p. 162, Mar. 2024, doi: 10.3390/info15030162.

[86] F. Sobhanmanesh, A. Beheshti, N. Nouri, N. M. Chapparo, S. Raj, and R. A. George, "A cognitive model for technology adoption," *Algorithms*, vol. 16, no. 3, p. 155, Mar. 2023, doi: 10.3390/a16030155.

[87] H. Kim, J.-H. Choi, and J.-Y. Choi, "A novel scheme for generating context-aware images using generative artificial intelligence," *IEEE Access*, vol. 12, pp. 31576–31588, 2024, doi: 10.1109/ACCESS.2024.3368871.

[88] G. Lan, S. Xiao, J. Yang, J. Wen, W. Lu, and X. Gao, "Active learning inspired method in generative models," *Expert Syst. Appl.*, vol. 249, Sep. 2024, Art. no. 123582, doi: 10.1016/j.eswa.2024.123582.

[89] P. Haindl and G. Weinberger, "Students' experiences of using ChatGPT in an undergraduate programming course," *IEEE Access*, vol. 12, pp. 43519–43529, 2024, doi: 10.1109/ACCESS.2024.3380909.

[90] S. Karpurapu, S. Myneni, U. Nettur, L. S. Gajja, D. Burke, T. Stiehm, and J. Payne, "Comprehensive evaluation and insights into the use of large language models in the automation of behavior-driven development acceptance test formulation," *IEEE Access*, vol. 12, pp. 58715–58721, 2024, doi: 10.1109/ACCESS.2024.3391815.

[91] J. Cámara, J. Troya, J. Montes-Torres, and F. J. Jaime, "Generative AI in the software modeling classroom: An experience report with ChatGPT and unified modeling language," *IEEE Softw.*, vol. 41, no. 6, pp. 73–81, Nov. 2024, doi: 10.1109/MS.2024.3385309.

[92] M. Deshpande and B. Magerko, "Holistic approach to design of generative AI evaluations: Insights from the research onion model," in *Proc. ACM IUI Workshops (Joint)*, 2024. [Online]. Available: https://ceur-ws.org/Vol-3660/paper10.pdf

[93] D. Kim, "Latent morphologies: Encoding architectural features and decoding their structure through artificial intelligence," *Int. J. Architectural Comput.*, vol. 22, no. 3, pp. 353–372, Sep. 2024, doi: 10.1177/14780771231209458.

[94] A. Bahi, J. Gharib, and Y. Gahi, "Integrating generative AI for advancing agile software development and mitigating project management challenges," *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 3, pp. 54–61, 2024, doi: 10.14569/ijacsa.2024.0150306.

[95] T. Braun and R. Möller, "AI in healthcare and the public sector: How to face the challenges of high-risk applications and what AI research can get out of it," *KI-Künstliche Intelligenz*, vol. 38, no. 3, pp. 119–126, Nov. 2024, doi: 10.1007/s13218-024-00853-w.

[96] M. La Salvia, E. Torti, E. Marenzi, G. Danese, and F. Leporati, "Edge and cloud computing approaches in the early diagnosis of skin cancer with attention-based vision transformer through hyperspectral imaging," *J. Supercomput.*, vol. 80, no. 11, pp. 16368–16392, Apr. 2024, doi: 10.1007/s11227-024-06076-y.

[97] S. Wang and Q. Li, "Remote collaborative process optimization in research and design of industrial manufacturing," *Auto. Intell. Syst.*, vol. 3, no. 1, Nov. 2023, Art. no. 10, doi: 10.1007/s43684-023-00056-4.

[98] R. C. Godwin, J. J. DeBerry, B. M. Wagener, D. E. Berkowitz, and R. L. Melvin, "Grant drafting support with guided generative AI software," *SoftwareX*, vol. 27, Sep. 2024, Art. no. 101784, doi: 10.1016/j.softx.2024.101784.

[99] K. T. Le and A. Andrzejak, "Rethinking AI code generation: A one-shot correction approach based on user feedback," *Automated Softw. Eng.*, vol. 31, no. 2, p. 60, Nov. 2024, doi: 10.1007/s10515-024-00451-y.

[100] A. Fitriawijaya and T. Jeng, "Integrating multimodal generative AI and blockchain for enhancing generative design in the early phase of architectural design process," *Buildings*, vol. 14, no. 8, p. 2533, Aug. 2024, doi: 10.3390/buildings14082533.

[101] A. Bedington, E. F. Halcomb, H. A. McKee, T. Sargent, and A. Smith, "Writing with generative AI and human-machine teaming: Insights and recommendations from faculty and students," *Comput. Composition*, vol. 71, Mar. 2024, Art. no. 102833, doi: 10.1016/j.compcom.2024.102833.



JOSE VALDIVIA received the B.S. degree in systems engineering from the University of Lima, Lima, Peru, and the M.S. degree in software engineering from the National University of San Marcos, Lima. He is currently pursuing the Ph.D. degree in systems engineering with the National University of San Marcos, Lima.

He has professional experience in web and mobile software development within enterprise environments and higher education institutions. He has led and managed software development teams using agile methodologies. His research interests include software engineering, artificial intelligence, and entrepreneurship.



DAVID MAURICIO received the B.S. degree in computer science from the National University of San Marcos, Lima, Peru, and the M.S. degree in applied mathematics and the Ph.D. degrees in computer systems and engineering from the Federal University of Rio de Janeiro, Rio de Janeiro, Brazil.

He was a Professor with the State University of Northern Rio de Janeiro, Brazil, from 1994 to 1998. Since 1998, he has been a Professor with the National University of San Marcos, Lima, Peru. His research interests include mathematical programming, artificial intelligence, software engineering, and entrepreneurship.



ROSARIO GUZMÁN-JIMÉNEZ (Senior Member, IEEE) received the B.S. degree in systems engineering from the Universidad Católica de Santa María, Arequipa, Peru, and the M.S. degree in systems engineering from the Universidad de Lima, Lima, Peru. She is currently pursuing the Ph.D. degree in education with the Universidad Femenina del Sagrado Corazón, Lima, and engineering studies with Pontificia Universidad Católica del Perú, with a focus on applied artificial intelligence.

She has more than 20 years of experience in higher education and is a Professor of systems engineering with expertise in database systems, business processes, ERP systems, and artificial intelligence. She is also a Research Professor with the Universidad de Lima, with publications related to the Yupana Inca Tawa Pukllay method and AI. Her research interests include ancestral numerical systems, computational thinking, neuroeducation, and social innovation.